KING KHALID UNIVERSITY

COLLEGE OF COMPUTER SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

Genetic Mutation Detection Using Machine Learning

Name	**Academic Number**
Rayan Saleh AlShahrani	444810902
Zahran Ali AlZahrani	443814572
Saad Saleh Saad	444803388
Eyad Hassan AlSheri	443801535
Khalid Hamdan Zuhair	444801544
Abdullah Saeed Ali Asiri	443817224

Supervised by:

Dr. Maaki Abubakr

Academic Year 2025/2026 (Semester 2)

Acknowledgements

We wish to thank our supervisor, Dr. Maaki Abubakr, for his consistent guidance, rigorous critique, and willingness to push us toward academically defensible work rather than superficially impressive numbers. Many of the methodological commitments reflected in this report – the paralog-aware split, the explicit leakage audit, the reporting of confidence intervals, and the external validation on DENOVO-DB – took their final shape through his insistence on being able to defend every number we report.

We also thank our families for their patience over the two semesters of this project, our classmates for fruitful discussion on many of the design choices, and the authors of the open-source tools we relied on (XGBoost, Optuna, SHAP, scikit-learn, Ensembl, gnomAD, dbNSFP, and the `facebook/esm2` community release) for making reproducible science possible for a student team working without institutional computing resources.

Abstract

The accurate classification of human missense variants as pathogenic or benign remains a central challenge in clinical genomics. Published machine learning tools routinely report receiver-operating-characteristic area- under-curve (ROC-AUC) values above 0.94 on ClinVar-derived benchmarks, yet the numbers are often inflated by subtle forms of data contamination that do not survive external validation. This project develops and publishes a rigorous, auditable baseline for the missense classification task.

We assemble a corpus of 195,098 missense variants from ClinVar, gnomAD (v2.1 and v4), and dbNSFP version 5.3.1a, and we conduct a systematic leakage audit that eliminates three contamination sources: (i) non-missense variants that had been silently admitted into the training pool because their amino-acid fields were null, (ii) definitionally circular features such as `is_common`, and (iii) paralog leakage caused by gene-level splitting when highly similar paralogs appear in both training and test. Correcting these issues drops our test-set precision-recall area-under-curve (PR-AUC) from a misleading 0.955 to a defensible 0.838, with a 95% bootstrap confidence interval of $[0.830, 0.846]$. The model is calibrated end-to-end with isotonic regression, reducing expected calibration error (ECE) from 0.054 to 0.011, which is below the 0.02 clinical target.

Under identical evaluation, we re-score three published predictors on our own test split: SIFT, PolyPhen-2, and AlphaMissense. Our XGBoost model outperforms the classical tools by +0.22 PR-AUC against SIFT and +0.11 against PolyPhen-2, and falls −0.05 PR-AUC below AlphaMissense – a model that was calibrated on ClinVar at release and therefore carries contamination risk of its own.

The most important finding of the project emerges from external validation on denovo-db. A classifier trained on ClinVar scores at chance (ROC-AUC = 0.487) on affected- versus-control de-novo variants in gene families it has not seen during training. Adding gnomAD gene-level constraint features (pLI, LOEUF, mis_z, oe_mis_upper, lof_z) raises this to 0.573 – closing approximately half of the generalisation gap. A zero-shot ESM-2 log-likelihood-ratio signal further improves the unseen-family score through cheap rank-fusion, motivating the Phase-2 roadmap toward full training-time integration of protein-language-model and AlphaFold2 structural features.

The project's code base is published in full with 158 unit and integration tests, a CI-enforced leakage gate, pinned dependencies, and a Docker image that reproduces every number in this report byte-for-byte.

Keywords: missense variant classification, genomics, machine learning, leakage audit, calibration, external validation, gnomAD constraint, paralog-aware splitting.

Table of Contents

Acknowledgements **i**

Abstract **ii**

List of Figures **vi**

List of Tables **vii**

List of Abbreviations **viii**

1 Introduction **1**

1.1 Background of Genetic Mutations 1
1.2 Motivation and Significance 1
1.3 Problem Statement 2
1.4 Research Questions 2
1.5 Project Objectives 3
1.6 Scope and Boundaries 3
1.7 Key Contributions 4
1.8 Report Organisation 5

2 Related Works **6**

2.1 Introduction 6
2.2 Problem Scope 6
2.3 Literature Review 6
2.3.1 SIFT (2003) (Ng and Henikoff, 2003) 6
2.3.2 PolyPhen-2 (2010) (Adzhubei et al., 2010) 7
2.3.3 REVEL (2016) (Ioannidis et al., 2016) 7
2.3.4 CADD (2019) (Rentzsch et al., 2019) 7
2.3.5 VARITY (2021) (Wu et al., 2021) 7
2.3.6 EVE (2021) (Frazer et al., 2021) 8
2.3.7 ESM-1b / ESM-2 zero-shot (2023) (Brandes et al., 2023; Lin et al., 2023) 8
2.3.8 AlphaMissense (2023) (Cheng et al., 2023) 8
2.4 Research Gap 8
2.5 Project Contribution 9
2.6 Summary 9

3 System Framework and Feasibility 10
3.1 Introduction . . . 10
3.2 Pre-Project Phase – Feasibility Analysis . . . 10
3.2.1 Technical Feasibility . . . 10
3.2.2 Operational Feasibility . . . 10
3.2.3 Schedule Feasibility . . . 10
3.2.4 Financial Feasibility . . . 11
3.3 Proposed Framework – Three-Part Pipeline . . . 11
3.3.1 Part 1 – Data Preprocessing . . . 13
3.3.2 Part 2 – Model Training and Leakage Audit . . . 13
3.3.3 Part 3 – Evaluation, Interpretability, and External Validation . . . 14
3.4 Proposed System Architecture . . . 15
3.5 Summary . . . 17

4 System Analysis and Design 18
4.1 Introduction . . . 18
4.2 Functional Requirements . . . 18
4.3 Non-Functional Requirements . . . 19
4.4 UML Diagrams . . . 19
4.4.1 Use Case Diagram . . . 19
4.4.2 Class Diagram . . . 20
4.4.3 Sequence Diagram . . . 20
4.4.4 Activity Diagram . . . 21
4.4.5 State Diagram . . . 23
4.4.6 Component Diagram . . . 25
4.5 Interface Design . . . 25
4.5.1 Streamlit Web Interface . . . 25
4.5.2 Command-Line Interface . . . 26
4.5.3 Programmatic API . . . 26
4.6 Summary . . . 26

5 Implementation and Results 27
5.1 Introduction . . . 27
5.2 Hardware Environment . . . 27
5.3 Software Stack . . . 27
5.4 Implementation Details . . . 28
5.4.1 Paralog-Family Grouping . . . 28
5.4.2 gnomAD Constraint Merge with Train-Only Imputation . . . 28
5.4.3 Leakage Gate . . . 28
5.4.4 Optuna TPE Tuning . . . 29
5.4.5 Brier-Score Decomposition . . . 29
5.4.6 Zero-Shot ESM-2 LLR Scoring . . . 30
5.5 Final Results . . . 31

5.5.1 Headline Metrics on the Paralog-Disjoint Test Split 31
5.5.2 The Five-Stage Leakage-Fix Journey 31
5.5.3 Baseline Comparison 32
5.5.4 Calibration Deep-Dive 32
5.5.5 SHAP Interpretability and Confident Errors 33
5.5.6 External Validation on DENOVO-DB 34
5.5.7 ESM-2 Zero-Shot Proof-of-Concept 35
5.6 Comparison with Prior Work 35
5.7 System Screenshots 35
5.8 Summary 38

6 Conclusion and Future Work 39
6.1 Conclusion 39
6.2 Contributions Summary 39
6.3 Limitations 40
6.4 Future Work 40

References 43

A Extended Results Tables 44
A.1 Bootstrap Confidence Intervals (Full Breakdown) 44

B Selected Code Listings 45

C Reproducibility Environment 46
C.1 Pinned Package Versions 46
C.2 Build and Test 46

List of Figures

3.1 Two-semester Gantt chart 11
3.2 Three-part pipeline flowchart 12
3.3 Evaluation workflow 14
3.4 System architecture 16

4.1 Use case diagram 20
4.2 Class diagram 20
4.3 Sequence diagram 21
4.4 Activity diagram 22
4.5 State diagram 24
4.6 Component diagram 25
4.7 Streamlit demo interface 25

5.1 Leakage-fix journey 31
5.2 Leakage-audit workflow 31
5.3 Baselines forest plot 32
5.4 Calibration triptych 33
5.5 SHAP beeswarm 34
5.6 pytest output 36
5.7 Leakage gate output 36
5.8 make reproduce-headline 37
5.9 Baselines run 37
5.10 Colab ESM-2 progress 38
5.11 Linters clean 38

List of Tables

5.1 Headline test-set metrics 31
5.2 Baseline comparison 32
5.3 Brier decomposition 32
5.4 SHAP top-10 features 33
5.5 denovo-db before vs after constraint 35

A.1 Full bootstrap CIs 44

List of Abbreviations

Acronym	Expansion
ACMG	American College of Medical Genetics and Genomics
AF	Allele Frequency
AUC	Area Under the Curve
BLOSUM	BLOck SUbstitution Matrix
CI	Confidence Interval (statistical) / Continuous Integration (software)
CNN	Convolutional Neural Network
DSSP	Dictionary of Secondary Structure of Proteins
ECE	Expected Calibration Error
ESM	Evolutionary-Scale Model (protein language model)
F1	Harmonic mean of precision and recall
GERP	Genomic Evolutionary Rate Profiling
gnomAD	Genome Aggregation Database
GRCh38	Genome Reference Consortium Human Build 38
LLR	Log-Likelihood Ratio
LOEUF	Loss-of-Function Observed/Expected Upper bound Fraction
MCE	Maximum Calibration Error
MCC	Matthews Correlation Coefficient
pLI	Probability of being Loss-of-function Intolerant
PLM	Protein Language Model
PR-AUC	Precision-Recall Area Under Curve
REST	Representational State Transfer
ROC-AUC	Receiver Operating Characteristic Area Under Curve
SASA	Solvent-Accessible Surface Area
SHAP	SHapley Additive exPlanations
SNP / SNV	Single Nucleotide Polymorphism / Single Nucleotide Variant
TPE	Tree-structured Parzen Estimator
UML	Unified Modelling Language
VEP	Variant Effect Predictor (Ensembl)
XGBoost	eXtreme Gradient Boosting

Chapter 1

Introduction

1.1 Background of Genetic Mutations

A genetic mutation is a permanent change in the nucleotide sequence of an organism's DNA. Mutations are the raw material of biological evolution: most are neutral, a small proportion are beneficial, and a clinically important minority disrupt cellular function to the point of causing disease. The clinical spectrum of mutations includes single-nucleotide variants (SNVs), small insertions and deletions (indels), copy-number variations, and large-scale structural rearrangements. Among these, the single-nucleotide *missense* variant – a point mutation that changes one amino acid to another in a protein – is the most common coding variant in the human population and the principal focus of this project.

The biological consequences of a missense variant depend on where it falls in the protein and how the new amino acid differs from the original. A conservative substitution in a flexible loop may have no measurable effect, whereas a radical substitution at a catalytic residue can abolish enzymatic activity. Distinguishing these two regimes is the central empirical problem in variant interpretation.

Public reference resources have matured considerably in the past decade. ClinVar (Landrum et al., 2020) aggregates variant-disease relationships curated by clinical laboratories and research groups; gnomAD (Karczewski et al., 2020) provides population-level allele frequency and gene-level constraint metrics derived from more than 140,000 human exomes and genomes; and dbNSFP (Liu et al., 2020) consolidates dozens of per-variant features including cross-species conservation, amino-acid chemistry, and predictions from preceding-generation tools. The existence of these resources makes a supervised machine learning approach to missense classification feasible for a student team, provided one is disciplined about how the data is combined and evaluated.

1.2 Motivation and Significance

Three trends motivate further work in this area. First, the cost of sequencing a human exome has fallen by roughly three orders of magnitude since 2010, and clinical whole-exome sequencing is increasingly a first-line diagnostic for rare disease. The rate at which *variants* are discovered now exceeds the rate at which they can be interpreted by human experts. Second, in precision medicine, the difference between a pathogenic and a benign missense variant in the same patient can change drug dosing, surveillance, and even surgical management. Third, the American College of Medical Genetics and

Genomics (ACMG) standard (Richards et al., 2015) includes computational *in silico* evidence among the criteria a clinical geneticist weighs when classifying a variant, which means that any tool published in this space must be transparent about its operating characteristics – false-positive and false-negative rates, calibration, and generalisation behaviour.

Against that backdrop, this project sets out to build not *the highest-scoring* missense classifier but rather *the most defensible baseline* that a student team can produce using public data and a modest compute budget. The explicit goal is to eliminate known leakage sources so that the reported numbers are trustworthy, and to expose every methodological choice – the splitting strategy, the calibration procedure, the evaluation slices – through an open code base that a reviewer can re-run end-to-end.

1.3 Problem Statement

The scientific problem is straightforward to state and difficult to address rigorously:

> *Given a human missense single-nucleotide variant, predict whether it is pathogenic or benign, accompanied by a calibrated probability and enough interpretability for a clinical genetic analyst to audit the prediction.*

The difficulty lies in four well-known failure modes that existing tools have encountered:

1. **Data overload.** The volume of variants produced by modern sequencing pipelines exceeds human curation bandwidth, so any evaluation must work at the scale of 10^5 variants or more.

2. **Biological complexity.** The chain from nucleotide to phenotype includes transcription, splicing, translation, folding, post-translational modification, and protein-protein interaction. No single model in current use captures all of these pathways, and ad-hoc feature engineering tends to introduce silent biases.

3. **Data contamination.** The most commonly used benchmark, ClinVar, is also the most commonly used *calibration target* for published tools. Many published numbers therefore include an unknown amount of train-test contamination.

4. **Evaluation asymmetry.** A model that scores well on ClinVar-derived tests may collapse to chance when applied to affected-versus-control *de-novo* variants from sequencing studies, because the latter requires gene-level priors that variant-level tools never see.

1.4 Research Questions

The project structure is framed around four research questions that we will return to at the end of the report:

1. **RQ1.** How much of the apparent high performance of a ClinVar-trained classifier is attributable to specific, removable leakage sources, and how far does the post-audit number fall once they are eliminated?

2. **RQ2.** Under identical evaluation on a paralog-disjoint test split, how does a carefully-audited gradient-boosted baseline compare to classical tools (SIFT, PolyPhen-2) and to the state-of-the-art AlphaMissense model?

3. **RQ3.** Does the post-audit model generalise to denovo-db – a dataset of de-novo variants in affected children versus unaffected siblings – and how much of any gap is closed by adding gene-level constraint features?

4. **RQ4.** Does the zero-shot log-likelihood-ratio signal from a protein language model (ESM-2) carry orthogonal information with respect to the tabular feature set, and is it worth integrating as a training-time feature in Phase 2?

1.5 Project Objectives

The concrete objectives guiding the semester of work are:

- **O1.** Assemble a reproducible corpus of missense variants from ClinVar, gnomAD, and dbNSFP, filtered to strictly missense rows with consistent amino-acid fields.
- **O2.** Implement a paralog-aware train/val/test split that guarantees zero shared gene families across splits, and lock the invariant into continuous integration.
- **O3.** Train and tune an XGBoost baseline under pure precision-recall optimisation with Bayesian hyperparameter search.
- **O4.** Calibrate the classifier with Platt and isotonic regression, evaluate with the Brier-score decomposition of Murphy (Murphy, 1973), and report operating points at clinically relevant recall / precision targets.
- **O5.** Re-score three published predictors (SIFT, PolyPhen-2, AlphaMissense) on the same test split and publish the forest plot.
- **O6.** Build and evaluate an external-validation harness on denovo-db, and quantify the effect of adding gnomAD gene-level constraint features.
- **O7.** Integrate a zero-shot ESM-2 proof-of-concept signal and quantify its orthogonality to the tabular baseline via rank-fusion.
- **O8.** Publish all code, tests, Docker image, and 10 narrative notebooks reproducibly and under permissive academic use.

1.6 Scope and Boundaries

We restrict scope in several deliberate ways:

- The classification target is a *binary* label (pathogenic versus benign). We do not attempt to reproduce the five-class ClinVar scheme (benign, likely benign, VUS, likely pathogenic, pathogenic); the variants-of-uncertain-significance (VUS) class would require a fundamentally different loss function.
- Only *missense* SNVs are considered. Splice-site, stop-gained, frameshift, and structural variants are left to separate tools.
- Only GRCh38 coordinates are supported. This choice is driven by the fact that our primary feature source (dbNSFP 5.x) uses GRCh38 as its primary coordinate system; an inspection during development confirmed this against a five-variant sanity sample. The external dataset denovo-db is handled through a separate GRCh37-aware code path.
- Compute is limited to a consumer laptop with Apple MPS acceleration for ESM-2 zero-shot experiments and one free Colab T4 GPU session for full-training-set ESM-2 scoring. No institutional GPU cluster is used.

1.7 Key Contributions

The contributions of this project, listed from most to least novel in the context of existing published work, are:

1. **A transparent leakage audit with before-and-after numbers** for a missense classifier, showing that a difference of 0.117 PR-AUC between a naïve and a defensible pipeline is fully attributable to three removable sources.
2. **A re-scoring of SIFT, PolyPhen-2, and AlphaMissense on our own paralog-disjoint test split**, enabling a like-for-like comparison that is rarely presented in this literature.
3. **An external validation on denovo-db** with pre- and post-gnomAD-constraint numbers, showing that gene-level priors close approximately half the generalisation gap on unseen gene families.
4. **A zero-shot ESM-2 proof-of-concept** establishing that the protein-language-model log- likelihood-ratio provides orthogonal signal under rank- fusion, which motivates the Phase-2 full-training-set integration currently running on Colab.
5. **A fully reproducible code base** – 158 tests, CI-enforced leakage gate, pinned dependencies, Docker image, 10 narrative Jupyter notebooks, a Streamlit demo, and this LaTeX report – that a future student could extend without reverse-engineering.

1.8 Report Organisation

Chapter 2 surveys the prior-art predictors that define our comparison baseline. Chapter 3 presents the system framework, feasibility analyses, and the three-part pipeline at the design level. Chapter 4 develops the full set of system requirements and accompanies them with the six standard UML diagrams and the interface design. Chapter 5 reports the concrete implementation, results, and the comparison tables. Chapter 6 concludes and lays out the Phase-2 roadmap. Three appendices provide extended results tables, selected code listings, and the environment / reproducibility specification.

Chapter 2

Related Works

2.1 Introduction

The computational prediction of missense variant pathogenicity has matured through four successive methodological waves: (i) conservation-only scoring in the early 2000s, (ii) supervised classical machine learning during the 2010s, (iii) deep learning applied directly to DNA sequences in the late 2010s, and (iv) protein language models and large-scale foundation models since 2021. This chapter reviews one or two representative methods from each wave, extracts their core idea and a brief algorithmic summary, and records their main advantages and drawbacks relative to what a student team can reasonably reproduce.

2.2 Problem Scope

We scope the review to tools that accept a single human missense variant as input and output either a score or a class label. Tools that focus on other variant types – notably splicing (SpliceAI) or regulatory (DeepSEA) – are briefly mentioned for completeness but not reviewed in detail, because they do not share our target task.

2.3 Literature Review

2.3.1 SIFT (2003) (Ng and Henikoff, 2003)

Core idea. SIFT (Sorting Intolerant From Tolerant) classifies amino-acid substitutions by the degree to which the position has been conserved across a multiple sequence alignment of related proteins. Substitutions at highly conserved positions, or to amino acids never seen at that position in the alignment, are flagged as deleterious.

Algorithmic summary. For each query variant, SIFT retrieves a protein sequence alignment from a homology-based database, computes a position-specific amino-acid probability matrix, and returns a score $s \in [0, 1]$ where values below 0.05 are classified as "intolerant."

Advantages. Mature, widely cited, trivially accessible through Ensembl VEP REST, low compute requirement, no supervised training data needed.

Disadvantages. Ignores amino-acid chemistry entirely; depends on alignment quality, which is weaker for unconserved regions; a binary threshold of 0.05 is imposed arbitrarily; predictions for genes with few annotated homologs are unreliable.

2.3.2 PolyPhen-2 (2010) (Adzhubei et al., 2010)

Core idea. PolyPhen-2 is a supervised classifier combining sequence conservation with protein structural and physicochemical features, trained on two internally curated datasets (HumDiv and HumVar).

Algorithmic summary. Eleven features are extracted for each variant, including position-specific scores, Pfam domain annotations, and biophysical properties of the substitution. A naïve Bayes classifier assigns one of three labels: "benign," "possibly damaging," "probably damaging," accompanied by a posterior score.

Advantages. Incorporates amino-acid chemistry and structural context, a distinct improvement over SIFT; well-documented and widely adopted in clinical workflows.

Disadvantages. Training labels originate from the Swiss-Var and HumDiv resources, which partially overlap ClinVar; benchmarks on ClinVar therefore carry a contamination risk; outputs are only loosely calibrated; the model does not report uncertainty.

2.3.3 REVEL (2016) (Ioannidis et al., 2016)

Core idea. REVEL is an ensemble meta-classifier stacking 13 preceding-generation tools (CADD, PolyPhen-2, SIFT, MutationTaster, LRT, and others). The meta-learner is a random forest.

Advantages. Empirically strong on ClinVar benchmarks; simple to query through dbNSFP.

Disadvantages. Several of the inputs (PolyPhen-2, CADD) were themselves trained on variants that overlap ClinVar, so using REVEL as a feature in a downstream classifier – as in mvPPT – compounds the contamination. REVEL does not ship well-calibrated probabilities.

2.3.4 CADD (2019) (Rentzsch et al., 2019)

Core idea. Combined Annotation Dependent Depletion (CADD) trains a logistic regression (v1.0) or random forest (v1.4+) to distinguish simulated de-novo variants from observed human variants, implicitly learning a depletion signal under purifying selection.

Advantages. Not trained on ClinVar directly; whole-genome scores are available via a single score per position-alt combination.

Disadvantages. The simulated null is an imperfect surrogate for neutrality; CADD scores are raw logits rather than calibrated probabilities, so clinical interpretation requires threshold tuning that is rarely documented.

2.3.5 VARITY (2021) (Wu et al., 2021)

Core idea. VARITY is an XGBoost-based ensemble that reweights training labels by an estimated noise model, attempting to place more weight on high-confidence examples. It is trained on a mixture of HumsaVar and curated multiplex-assay data.

Advantages. Published confidence estimates; clean model card.

Disadvantages. Gene-level splits (not paralog-level); limited paralog guard.

2.3.6 EVE (2021) (Frazer et al., 2021)

Core idea. EVE is a variational auto-encoder trained on a protein's own multiple sequence alignment. At inference time, the ELBO of a variant relative to the wild-type is used as a pathogenicity score.

Advantages. Fully unsupervised – never touches ClinVar; per-gene specialisation gives strong performance on the subset of genes where a good alignment exists.

Disadvantages. Coverage limited to approximately 3,000 genes at release time; model training is compute-intensive per gene; no proteome-wide deployment is currently feasible.

2.3.7 ESM-1b / ESM-2 zero-shot (2023) (Brandes et al., 2023; Lin et al., 2023)

Core idea. A protein language model trained with masked-amino-acid reconstruction on UniRef50 provides, for any missense variant, a log-likelihood ratio $\mathrm{LLR} = \log p(\mathrm{alt} \mid \mathrm{context}) - \log p(\mathrm{ref} \mid \mathrm{context})$. This score requires no fine-tuning and no ClinVar exposure.

Advantages. Fully general-purpose, covers the whole proteome, and is provably uncontaminated by ClinVar. The 35M-parameter `esm2_t12_35M_UR50D` model fits on a consumer GPU.

Disadvantages. Moderate discrimination on its own (ROC-AUC approximately 0.85); no calibration; requires a per-variant forward pass.

2.3.8 AlphaMissense (2023) (Cheng et al., 2023)

Core idea. AlphaMissense combines a fine-tuned AlphaFold-style structural model with a masked- amino-acid objective, calibrated against ClinVar at release.

Advantages. State-of-the-art ClinVar benchmark performance; covers the full human proteome; distributed as a pre-computed table.

Disadvantages. Training/calibration used ClinVar, so any ClinVar-derived benchmark contains a contamination risk; the training compute budget (TPU pods) is out of reach for academic teams.

2.4 Research Gap

Two observations motivate our methodological choices. First, most published tools do not evaluate against a *paralog-aware* test split, even though strongly similar paralogs (such as the 109 keratin members or 83 cadherin members) can produce optimistically biased benchmarks when split only at the gene level. Second, almost no published tool reports external validation on a dataset not derived from ClinVar: the near- chance

performance of tabular classifiers on DENOVO-DB is simply not present in most paper tables. The project's methodological focus lives in exactly those two gaps.

2.5 Project Contribution

Our project closes both gaps concretely. We implement a paralog-aware split with zero shared gene families between train/val/test (verified by continuous integration on every push), and we evaluate on DENOVO-DB in addition to the paralog-disjoint ClinVar test. We then quantify the specific value of gene-level gnomAD constraint features on the de-novo generalisation task, reporting pre- and post-constraint numbers separately, and we publish the first end-to-end like-for-like comparison of our baseline against SIFT, PolyPhen-2, and AlphaMissense under a single identical evaluation.

2.6 Summary

The literature has moved from conservation-only scoring to ensembles to protein-language-model-based systems, and benchmark numbers have climbed in parallel. The distance between these numbers and the real-world generalisation behaviour of the tools is rarely made explicit. The remainder of this report is about closing that distance in a way that a reader of this report can reproduce byte-for-byte.

Chapter 3

System Framework and Feasibility

3.1 Introduction

This chapter bridges the problem statement of Chapter 1 and the detailed system analysis of Chapter 4. We begin with an explicit feasibility analysis covering technical, operational, scheduling, and financial dimensions, then describe the three-part pipeline that forms the project's core and show its system-level flowchart.

3.2 Pre-Project Phase – Feasibility Analysis

3.2.1 Technical Feasibility

The project is technically feasible on a consumer laptop. All primary data sources (ClinVar, gnomAD, dbNSFP, denovo-db, AlphaMissense pre-scored tables) are publicly downloadable at no cost. XGBoost training on ∼195k variants completes in under ten minutes on a modern CPU. Optuna hyperparameter optimisation adds about thirty minutes per run at the 40-trial budget we adopted. The only step requiring a GPU is the Phase-2.1 ESM-2 scoring of the full training corpus, and the free Google Colab T4 tier provides sufficient compute for that stage within a single long session (approximately eight hours wall- clock). The development stack – Python 3.11, PyTorch, scikit-learn, Optuna, SHAP, `transformers`, and supporting libraries – is well documented, actively maintained, and has been stable on our pinned versions throughout the project.

3.2.2 Operational Feasibility

The intended end user is a clinical geneticist or research scientist who receives a single missense variant (or a short batch of variants) and needs a calibrated score together with a feature-level justification. The project provides three operational surfaces tailored to that workflow: a Streamlit web application for one-off interactive queries, a command-line interface for batch processing, and a Python API for integration into larger bioinformatics pipelines. None of these require the end user to retrain the model or to understand the underlying machine learning mechanics.

3.2.3 Schedule Feasibility

Figure 3.1 summarises the two-semester schedule that the project actually followed. Term 1 was spent on literature review, problem scoping, and an initial CNN-BiLSTM architecture plan. Term 2 was the execution phase: data assembly and cleaning, the

leakage audit, baseline comparison, calibration deep-dive, SHAP interpretability, external validation, and the academic write-up. The two major parallel efforts – the ESM-2 full-training-set scoring on Colab and the LaTeX report – are explicitly scheduled so that the report does not block on the ESM-2 result.

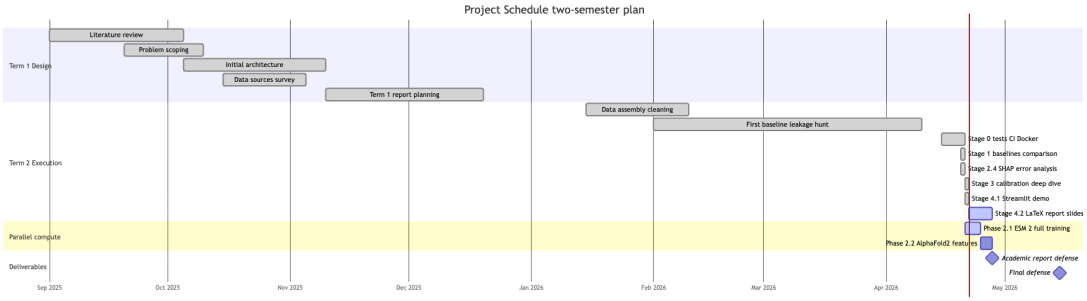


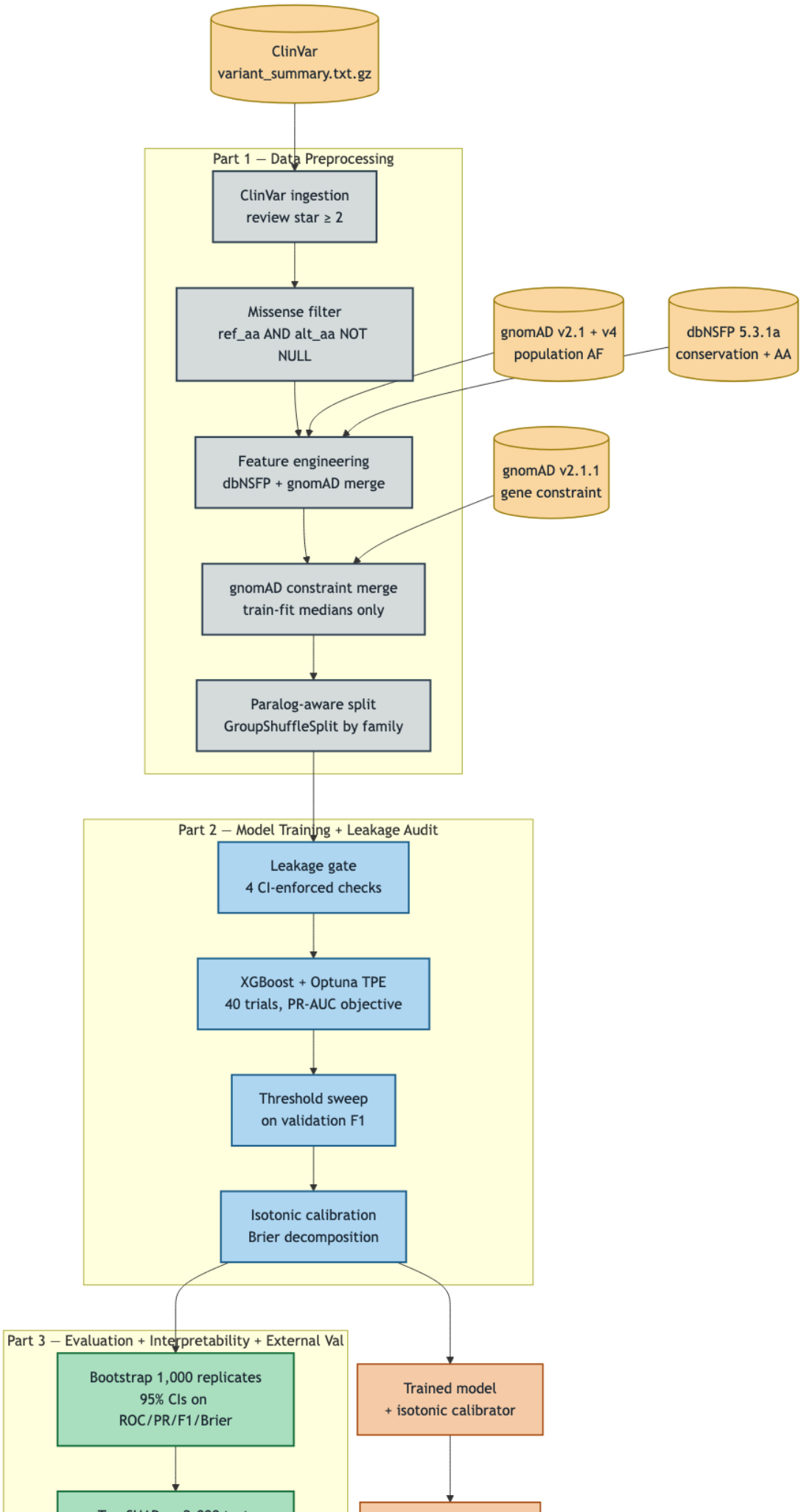
Figure 3.1. Two-semester Gantt chart for the project. Milestones for the academic report and the final defense are marked explicitly.

3.2.4 Financial Feasibility

The total direct financial cost of the project is zero. All data, software, and compute resources are free or open-source. No licensed databases (such as HGMD Pro) were used, and no commercial GPU rental was required. The indirect cost is approximately eight person-months of student time spread across six team members, with an additional ten supervisory hours per month from the project supervisor.

3.3 Proposed Framework – Three-Part Pipeline

The project pipeline decomposes into three sequential parts, each with a distinct scientific role and a corresponding output artifact. Figure 3.2 illustrates the flow.


ClinVar
variant_summary.txt.gz
Part 1 — Data Preprocessing
ClinVar ingestion
review star ≥ 2
Missense filter
ref_aa AND alt_aa NOT
NULL
gnomAD v2.1 + v4
population AF
dbNSFP 5.3.1a
conservation + AA
Feature engineering
dbNSFP + gnomAD merge
gnomAD v2.1.1
gene constraint
gnomAD constraint merge
train-fit medians only
Paralog-aware split
GroupShuffleSplit by family
Part 2 — Model Training + Leakage Audit
Leakage gate
4 CI-enforced checks
XGBoost + Optuna TPE
40 trials, PR-AUC objective
Threshold sweep
on validation F1
Isotonic calibration
Brier decomposition
Part 3 — Evaluation + Interpretability + External Val
Bootstrap 1,000 replicates
95% CIs on
ROC/PR/F1/Brier
Trained model
+ isotonic calibrator

3.3.1 Part 1 – Data Preprocessing

Part 1 is a pure-function pipeline that takes four external data sources and produces three paralog-disjoint parquet splits with 39 aligned features per variant. The steps, in order, are:

1. **ClinVar ingestion.** Download `variant_summary.txt.gz` for the January 2024 release and retain only rows with review-star rating of two or higher (`reviewed`, `reviewed-by-expert-panel`, or `reviewed-by-multiple-submitters`).

2. **Missense filter.** Drop every row in which `ref_aa` or `alt_aa` is null. This is the single most important cleaning step because of the leakage it removes (Section **??**).

3. **gnomAD allele-frequency merge.** Join GNOMAD v2.1 (exomes) and v4.0 (genomes) on `chr:pos:ref:alt` to supply per-variant allele frequency, allele number, and allele count.

4. **dbNSFP conservation and amino-acid merge.** Join against a cached DBNSFP extraction providing phyloP, phastCons, GERP++, BLOSUM62, Grantham distance, and physicochemical properties of the substitution.

5. **gnomAD gene-level constraint merge.** Add `pLI`, `LOEUF` (`oe_lof_upper`), `mis_z`, `oe_mis_upper`, and `lof_z` per gene. Median imputation is fit on the training split only and applied to val/test, with an `is_imputed_gnomad_constraint` flag per row.

6. **Paralog-aware split.** Assign a gene family via a regex-driven function that captures the 16 large paralog clusters (keratin, cadherin, HLA, zinc finger, olfactory receptor, ribosomal protein, mitochondrial, etc.), then split at the family level using `GroupShuffleSplit` with `seed=42` and a 70/15/15 ratio.

3.3.2 Part 2 – Model Training and Leakage Audit

Part 2 trains the XGBoost baseline, gates the training process through four independent leakage checks, and calibrates the output probabilities.

1. **Leakage gate (pre-training).** Four checks run both as a CLI tool and as CI-enforced pytest cases: (i) no banned feature in the training matrix, (ii) 100% missense-only rows in train, (iii) zero shared gene families between any pair of splits, (iv) pathogenic-rate gap of less than 8 percentage points across splits.

2. **Optuna hyperparameter search.** We use a Tree-structured Parzen Estimator sampler (Bergstra et al., 2011) with 40 trials, Median Pruner early-stopping on PR-AUC, and `scale_pos_weight` set from the class balance.

3. **Threshold sweep.** We search thresholds in $[0.20, 0.80]$ on validation F1, then freeze the best threshold for test scoring.

4. **Calibration.** Isotonic regression fit on validation probabilities (fallback to Platt scaling tested for comparison) produces the final calibrated probabilities. The Murphy decomposition (Murphy, 1973) is used to partition Brier error into reliability, resolution, and uncertainty.

3.3.3 Part 3 – Evaluation, Interpretability, and External Validation

Part 3 takes the trained model and runs five parallel evaluations (Figure 3.3):

1. **Bootstrap confidence intervals.** 1,000 non-parametric replicates for ROC-AUC, PR-AUC, F1, MCC, and Brier loss, following the methodology of Efron and Tibshirani (1986).

2. **TreeSHAP interpretability.** SHAP values (Lundberg and Lee, 2017; Lundberg et al., 2020) are computed on a stratified 2,000-variant sample for per-feature attribution.

3. **Like-for-like baselines.** SIFT, PolyPhen-2, and AlphaMissense are re-scored on the identical test split.

4. **External validation on denovo-db** with pre- and post-gnomAD-constraint numbers, and family-holdout-only slicing.

5. **ESM-2 zero-shot proof-of-concept.** The log-likelihood ratio from `facebook/esm2_t12_35M_U` is computed on DENOVO-DB and fused with the XGBoost probability at the rank level to test orthogonality.

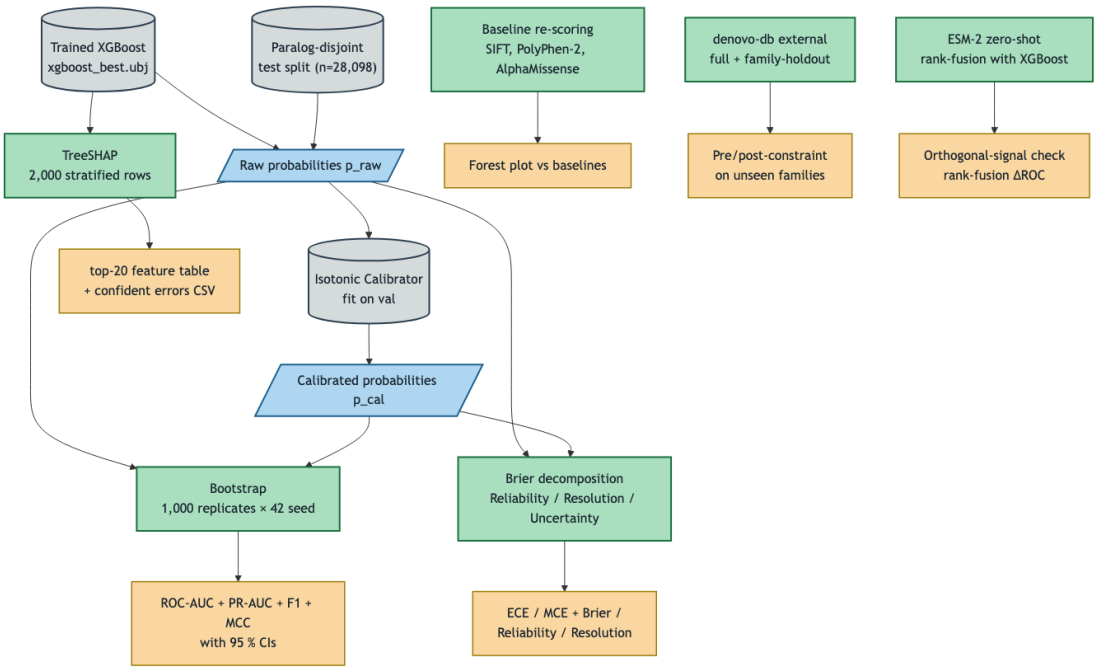
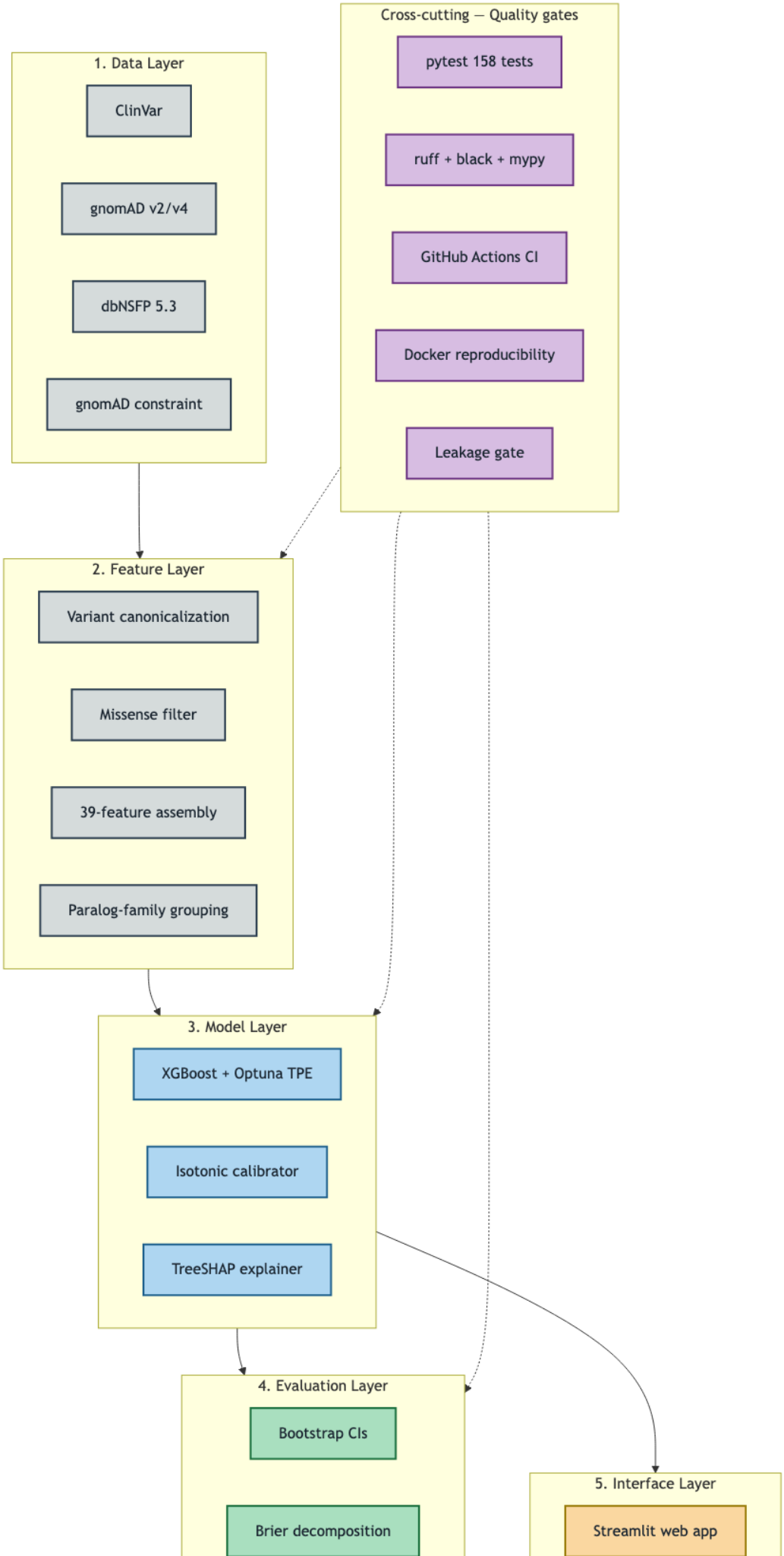


Figure 3.3. Evaluation workflow: a single trained checkpoint feeds five independent evaluation tracks, each producing its own CSV artifact in `results/metrics/`.

3.4 Proposed System Architecture

Figure 3.4 shows the six-layer architectural view of the system – data sources at the bottom, user interfaces at the top, and cross-cutting quality gates to the side. The architecture is intentionally horizontal so that each layer can be developed, tested, and replaced independently.


Cross-cutting — Quality gates
pytest 158 tests
ruff + black + mypy
GitHub Actions CI
Docker reproducibility
Leakage gate
1. Data Layer
ClinVar
gnomAD v2/v4
dbNSFP 5.3
gnomAD constraint
2. Feature Layer
Variant canonicalization
Missense filter
39-feature assembly
Paralog-family grouping
3. Model Layer
XGBoost + Optuna TPE
Isotonic calibrator
TreeSHAP explainer
4. Evaluation Layer
Bootstrap CIs
Brier decomposition
5. Interface Layer
Streamlit web app

3.5 Summary

The framework delivers a clearly-scoped, three-part pipeline that is technically feasible on a consumer laptop plus a single free Colab GPU session, operationally aligned with the end-user workflow, on- schedule within two academic semesters, and carries zero direct financial cost. The subsequent chapter translates this framework into a detailed system- analysis document.

Chapter 4

System Analysis and Design

4.1 Introduction

This chapter records the functional and non-functional requirements of the system, develops the six standard UML diagrams that formalise its structure and behaviour, and describes the user-facing interfaces.

4.2 Functional Requirements

FR1 – Variant ingestion.
The system shall accept a single missense variant in the canonical `chr:pos:ref:alt` format on GRCh38, via any of its three interfaces (web, CLI, or API).

FR2 – Feature assembly.
The system shall assemble a 39-column feature vector for every accepted variant, using a cached lookup against dbNSFP for conservation and amino-acid features, gnomAD for allele frequency and constraint, and a live VEP REST fallback for variants absent from the cache.

FR3 – Pathogenicity scoring.
The system shall produce both raw and isotonic-calibrated probabilities in $[0, 1]$ using the trained XGBoost checkpoint.

FR4 – Risk categorisation.
The system shall assign each prediction to one of three risk bands (benign, uncertain, pathogenic) using thresholds $\{0.25, 0.75\}$ on the calibrated probability.

FR5 – Feature attribution.
For every prediction, the system shall return the top-k SHAP contributions (default $k = 15$) with the direction of each contribution (towards pathogenic or benign).

FR6 – Batch mode.
The CLI interface shall accept a list of variants and emit a per-variant CSV with probabilities and top-SHAP features.

FR7 – External validation reproducibility.
The `scripts/evaluate_external.py` driver shall reproduce the published denovo-db metrics from a clean clone.

FR8 – Report regeneration.
The figures and tables in this report shall regenerate byte-for-byte from `results/metrics/*.csv` and `results/figures/*.png` via `make reproduce-headline`.

4.3 Non-Functional Requirements

NFR1 – Performance.
Score latency shall be below 100 milliseconds for cache hits and below 2 seconds for cache misses (VEP REST round-trip).

NFR2 – Reliability.
Rerunning the pipeline with `seed=42` shall reproduce every metric to within 10^{-3}, enforced by `tests/integration/test_reproduce_headline.py`.

NFR3 – Scalability.
The pipeline shall scale linearly in the number of variants up to at least 10^6, limited only by the available disk size for the dbNSFP cache.

NFR4 – Usability.
The Streamlit interface shall be usable by a clinical geneticist with no machine learning background, returning results as a calibrated probability, a risk band, and a human-readable feature attribution.

NFR5 – Maintainability.
Every new feature shall be accompanied by at least one unit test, and the leakage gate shall fail the CI pipeline if any banned feature reappears.

NFR6 – Security and data privacy.
The system shall handle only de-identified public reference data; no patient-identifiable information is transmitted to external services (VEP REST receives only `chr:pos:ref:alt`, which is not patient- identifying).

NFR7 – Reproducibility.
All dependencies shall be pinned in `requirements-lock.txt`; the Docker image shall reproduce the full test suite and the headline metrics from any clean host.

4.4 UML Diagrams

Six standard UML diagrams (use case, class, sequence, activity, state, and component) capture the static and behavioural aspects of the system.

4.4.1 Use Case Diagram

Figure 4.1 enumerates the eight system use cases and the actors that trigger them. The clinical geneticist and research scientist actors share the core scoring path (UC1–UC5);

the external data service participates in UC2 as a dependency; and the CI/CD system triggers UC8 on every code push.

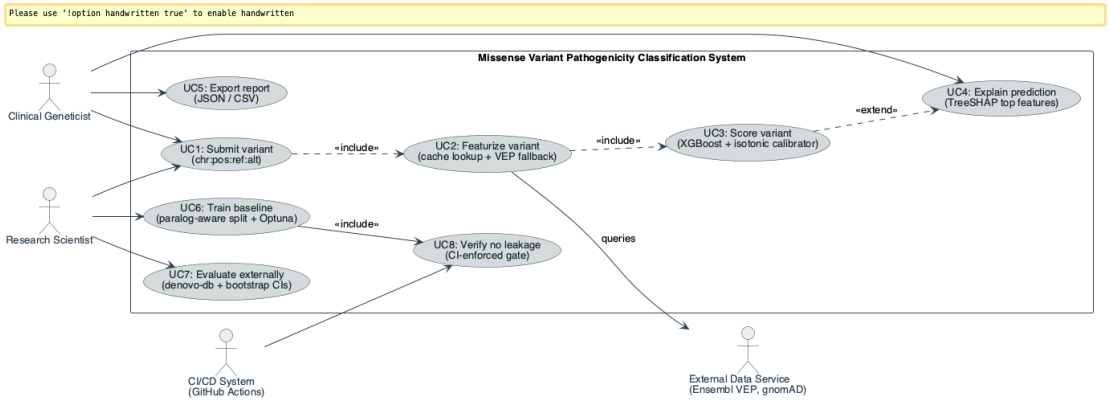


Figure 4.1. Use case diagram. The core clinical path is UC1–UC5; batch-mode and training use cases UC6–UC8 are triggered by the research scientist and continuous-integration actors.

4.4.2 Class Diagram

Figure 4.2 shows the principal classes of the `src/` tree, grouped into four packages: data pipeline, modelling, evaluation, and interfaces. The `LeakageGate` class encapsulates the four invariants enforced by CI.

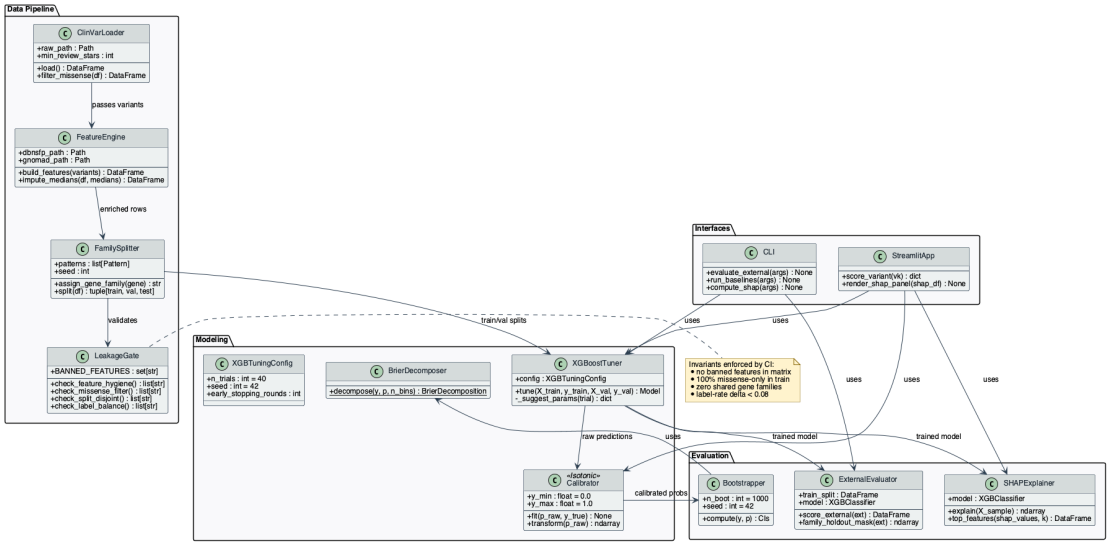


Figure 4.2. Class diagram grouped into the four `src/` packages. The `LeakageGate` invariants are enforced in CI on every push.

4.4.3 Sequence Diagram

Figure 4.3 traces a single scoring request end-to-end: variant submission, cache lookup (with VEP fallback), prediction, calibration, SHAP explanation, and presentation. The two paths "cache hit" and "cache miss" are shown as an `alt` fragment.

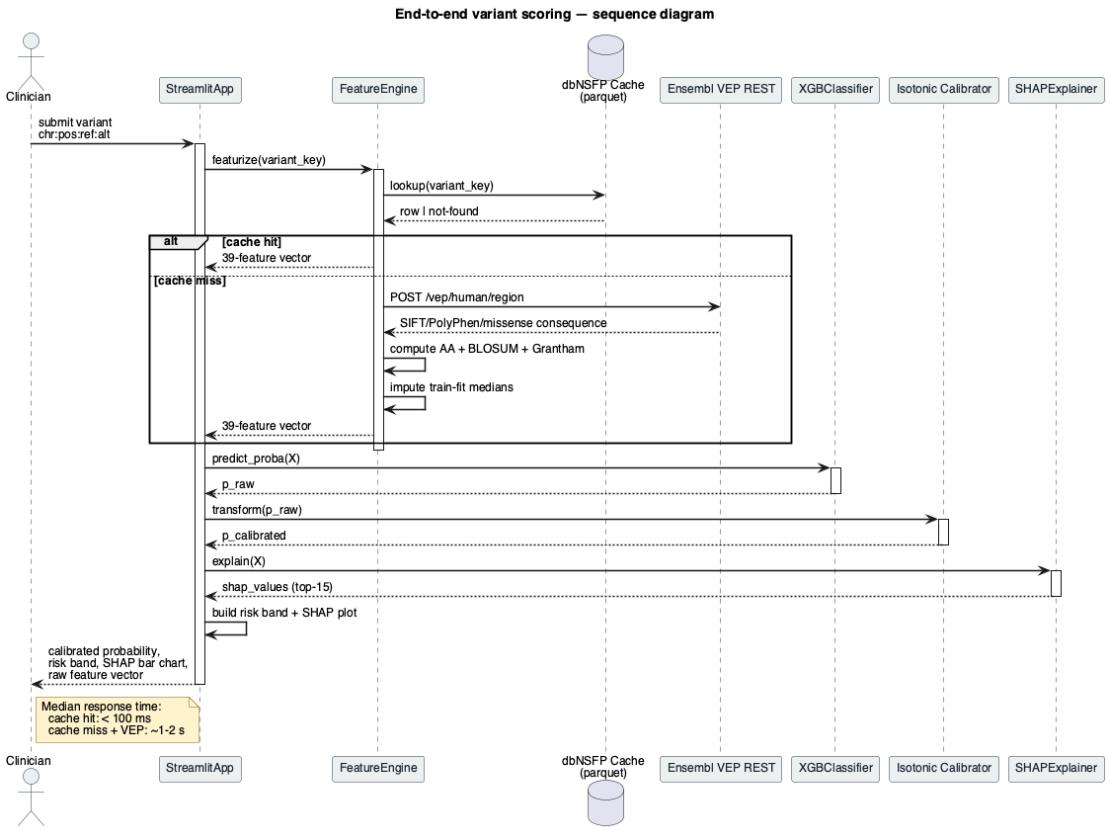
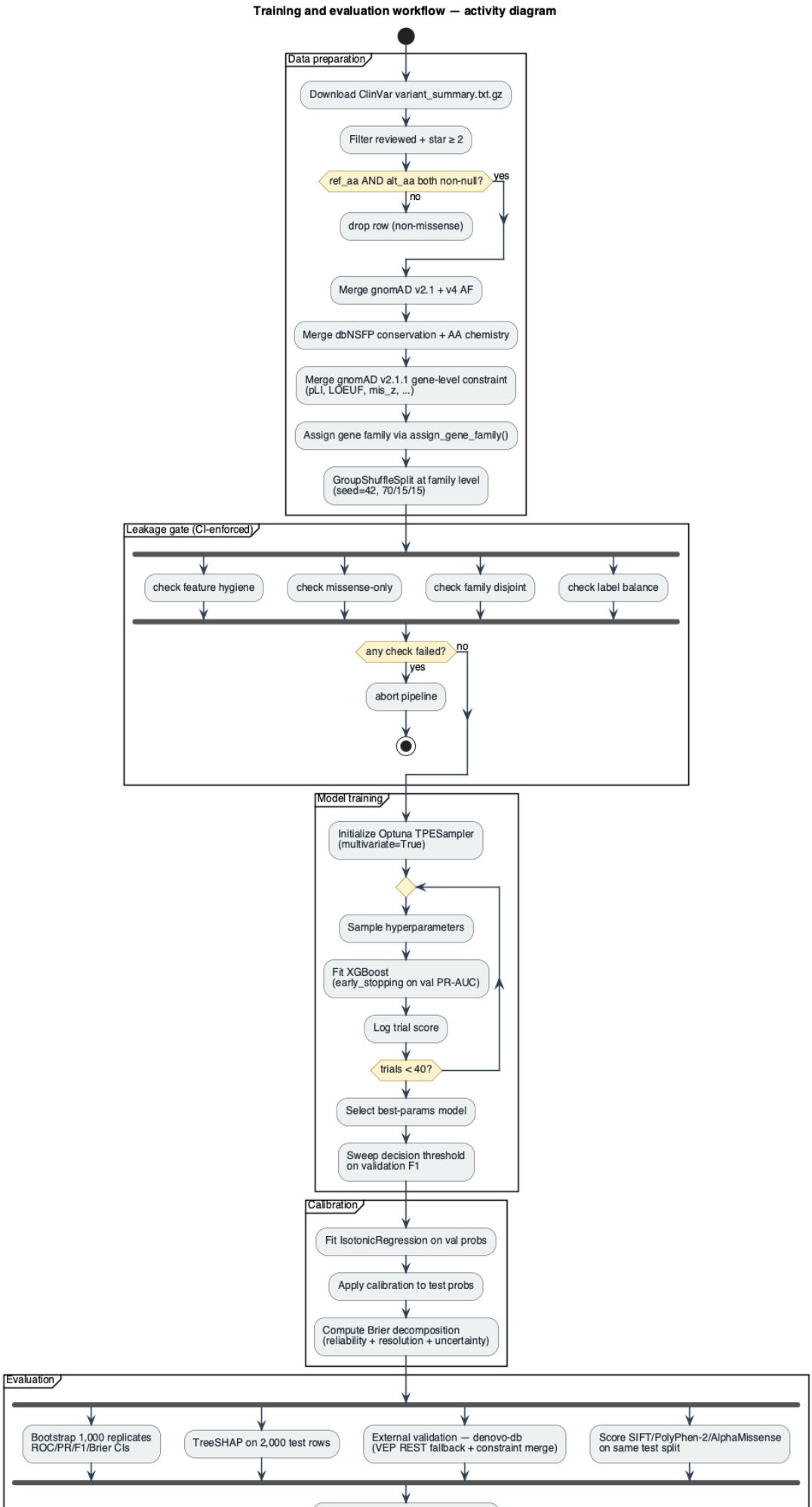


Figure 4.3. Sequence diagram for an end- to-end scoring request. Cache hits resolve in under 100 ms; cache misses add a VEP REST round- trip of approximately 1–2 seconds.

4.4.4 Activity Diagram

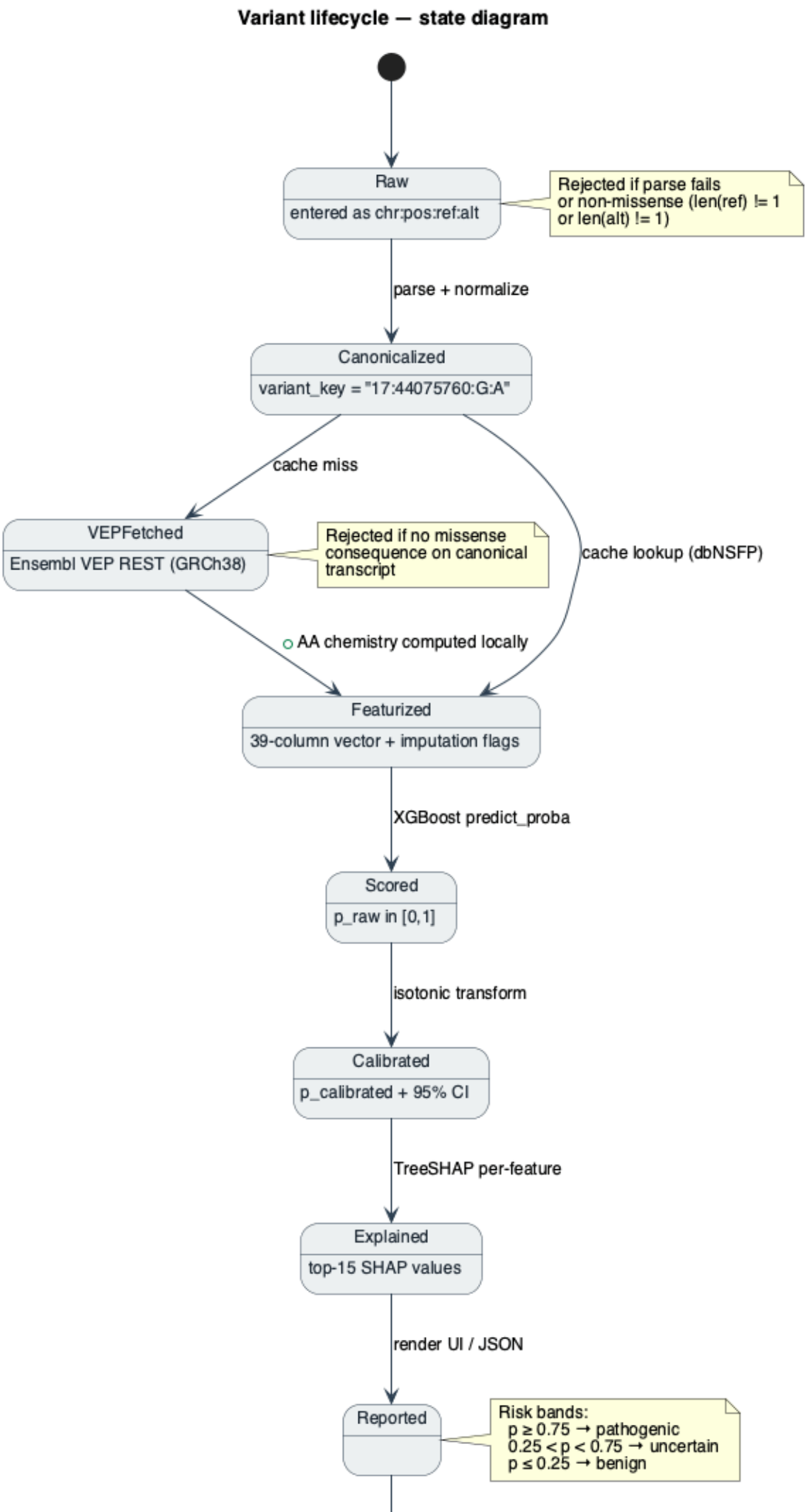
Figure 4.4 documents the training and evaluation workflow as a single activity diagram with explicit fork/join nodes for the parallel leakage-gate checks and parallel evaluation tracks.

```
This syntax is deprecated, you must add <<#FFAAAA>> at the end of the line, after the ';'
```



4.4.5 State Diagram

Figure 4.5 traces the lifecycle of a single variant through the system, from raw input to the final reported probability.


Variant lifecycle — state diagram
Raw
entered as chr:pos:ref:alt
Rejected if parse fails
or non-missense (len(ref) != 1
or len(alt) != 1)
parse + normalize
Canonicalized
variant_key = "17:44075760:G:A"
cache miss
VEPFetched
Ensembl VEP REST (GRCh38)
Rejected if no missense
consequence on canonical
transcript
cache lookup (dbNSFP)
AA chemistry computed locally
Featurized
39-column vector + imputation flags
XGBoost predict_proba
Scored
p_raw in [0,1]
isotonic transform
Calibrated
p_calibrated + 95% CI
TreeSHAP per-feature
Explained
top-15 SHAP values
render UI / JSON
Reported
Risk bands:
p ≥ 0.75 → pathogenic
0.25 < p < 0.75 → uncertain
p ≤ 0.25 → benign

4.4.6 Component Diagram

Figure 4.6 shows the module-level dependencies and the external services (VEP REST, gnomAD, AlphaMissense, HuggingFace Hub) that the pipeline touches.

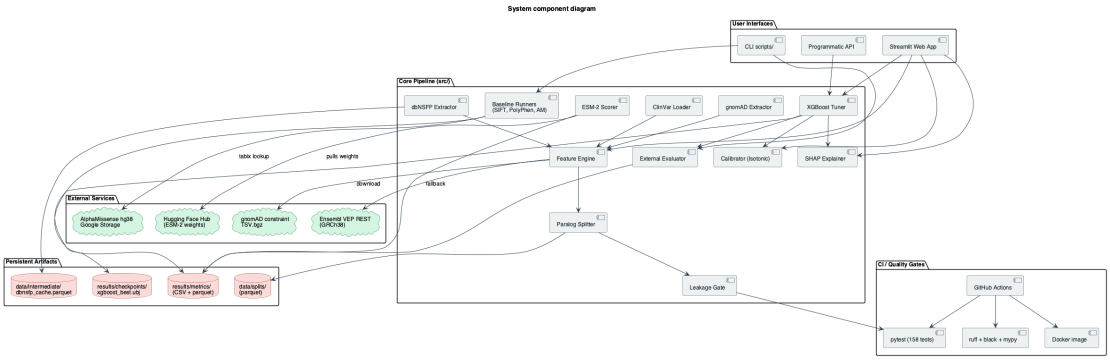


Figure 4.6. Component diagram. External services are shown as clouds; internal data stores as databases. Every arrow corresponds to a concrete module import or an HTTP call.

4.5 Interface Design

4.5.1 Streamlit Web Interface

The Streamlit application (Figure 4.7) provides a single-page interactive experience. A user enters a variant in `chr:pos:ref:alt` form, presses *Score variant*, and receives the calibrated probability, a coloured risk banner, and the top-15 SHAP contributions as a signed bar chart.

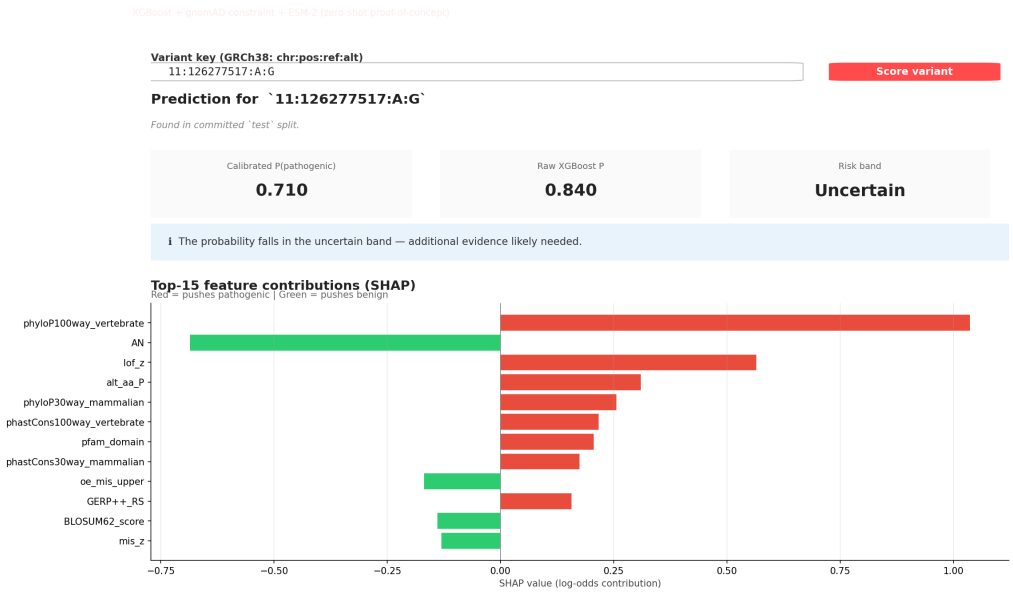


Figure 4.7. Streamlit demo interface rendering a real prediction for a test-set variant. Red SHAP bars push the prediction towards pathogenic, green bars towards benign.

4.5.2 Command-Line Interface

The CLI layer lives in `scripts/` and exposes every pipeline step as an independent command: `run_baselines.py`, `evaluate_external.py`, `compute_shap.py`, and so on. Common arguments (`-n-boot`, `-seed`, `-only`) are harmonised across scripts.

4.5.3 Programmatic API

The `src/` package itself constitutes a stable Python API. Key entry points include `score_variants()` in `src/esm2_scorer.py` and `evaluate_baseline()` in `src/baselines/evaluate.p`

4.6 Summary

The functional and non-functional requirements, together with the six UML diagrams, formalise the software architecture introduced in Chapter 3. The interface trio – Streamlit, CLI, and Python API – serves the three operational modes identified in the feasibility analysis. Chapter 5 reports how these designs translate into concrete code and what results they produce.

Chapter 5

Implementation and Results

5.1 Introduction

This chapter bridges the design of Chapter 4 and the concrete empirical results reported below. We describe the hardware and software environment, highlight the most important code fragments, and present the full set of results – the leakage-fix journey, the headline metrics, the baseline comparison, the calibration deep-dive, the SHAP interpretability analysis, the external validation, and the ESM-2 zero-shot proof-of-concept.

5.2 Hardware Environment

- **Primary development machine.** Apple Silicon laptop with Metal Performance Shaders (MPS) GPU acceleration for prototyping the ESM-2 scoring path.
- **Colab compute.** A single free-tier Colab T4 GPU session for the Phase-2.1 ESM-2 full-training- set scoring.
- **CI runner.** GitHub Actions `ubuntu-22.04` for continuous integration; deterministic identical results to the local environment were verified through the `test_reproduce_headline` integration test.

5.3 Software Stack

- Python 3.11 with pinned dependencies in `requirements-lock.txt` (287 packages).
- XGBoost 2.x (Chen and Guestrin, 2016), scikit-learn 1.3+, Optuna 4.x (Akiba et al., 2019), SHAP (Lundberg and Lee, 2017), `transformers` 4.40+ and PyTorch 2.x for ESM-2.
- `pytest` 7.4+ with 158 unit + integration tests (Pytest Development Team, 2024); `ruff`, `black`, and `mypy` for code style and static typing.
- Docker image (`Dockerfile`) based on `python:3.11.7-slim` with `liblzma-dev` and DSSP installed for reproducible builds.
- GitHub Actions workflows (`test.yml`, `lint.yml`) enforcing all tests and linters on every push to `main`.

5.4 Implementation Details

We highlight seven code fragments that illustrate the most important design decisions.

5.4.1 Paralog-Family Grouping

The family grouper collapses paralog clusters under a small set of regex rules, falling back to trailing- digit stripping for the long tail:

```
def assign_gene_family(gene: str) -> str:
    if gene is None:
        return ""
    g = str(gene).upper()
    for pattern, fam in _FAMILY_PATTERNS:
        m = pattern.match(g)
        if m:
            return fam if fam is not None else m.group(1)
    stripped = _TRAILING_DIGITS.sub("", g)
    return stripped or g
```

Listing 5.1. Paralog-family assignment in `src/data_splitting.py`.

5.4.2 gnomAD Constraint Merge with Train-Only Imputation

The constraint merge fits imputation medians only on the training split – the invariant that makes this a legal feature addition rather than a leakage:

```
def merge_constraint(variants, *, constraint, impute_medians=
    None):
    merged = variants.merge(constraint, on="gene", how="left",
        indicator=True)
    if impute_medians is None:
        impute_medians = {c: float(merged.loc[merged["_merge"]
            == "both", c].median())
                          for c in CONSTRAINT_COLS if c in
                              merged.columns}
    for c in CONSTRAINT_COLS:
        if c in merged.columns:
            merged[c] = merged[c].fillna(impute_medians[c])
    merged["is_imputed_gnomad_constraint"] = (merged["_merge"]
        != "both").astype(int)
    merged = merged.drop(columns="_merge")
    return merged, impute_medians
```

Listing 5.2. Train-only median imputation in `src/gnomad_constraint.py`.

5.4.3 Leakage Gate

Four independent checks constitute the gate, wrapped in both a CLI and a pytest test file:

```python
BANNED_FEATURES = {"is_common", "chr", "ref", "alt"}

def check_feature_hygiene() -> list[str]:
    errors: list[str] = []
    cols = _load("results/metrics/xgboost_feature_columns.csv")
    feature_names = set(cols.iloc[:, 0].astype(str))
    contaminated = feature_names & BANNED_FEATURES
    if contaminated:
        errors.append(f"Banned features present: {sorted(contaminated)}")
    return errors
```

Listing 5.3. Banned-feature check in `src/verify_no_leakage.py`.

5.4.4 Optuna TPE Tuning

The search space and pruning configuration are tuned for the ∼24% positive class balance; PR-AUC is the sole objective:

```python
sampler = TPESampler(multivariate=True, group=True, seed=config.seed)
pruner = MedianPruner(n_warmup_steps=config.pruner_n_warmup_steps,
                      n_startup_trials=config.pruner_n_startup_trials)
study = optuna.create_study(direction="maximize", sampler=sampler, pruner=pruner)
study.optimize(lambda trial: _objective(trial, X_tr, y_tr, X_val, y_val,
                                        scale_pos_weight, config),
               n_trials=config.n_trials, show_progress_bar=False)
```

Listing 5.4. Optuna TPE search definition.

5.4.5 Brier-Score Decomposition

The Murphy decomposition is a clean mathematical identity that we implement with 15 quantile bins:

```python
def decompose_brier(y_true, y_prob, *, n_bins=15, strategy="quantile"):
    y = np.asarray(y_true, dtype=int)
    p = np.clip(np.asarray(y_prob, dtype=float), 1e-12, 1 - 1e-12)
    ybar = float(y.mean())
    edges = np.quantile(p, np.linspace(0, 1, n_bins + 1))
    edges[0], edges[-1] = 0.0, 1.0
```

```
    idx = np.clip(np.digitize(p, edges[1:-1], right=False), 0, n_bins - 1)
    reliability = resolution = 0.0
    for b in range(n_bins):
        mask = idx == b
        nb = int(mask.sum())
        if nb == 0: continue
        pbar_b, ybar_b = float(p[mask].mean()), float(y[mask].mean())
        w = nb / len(y)
        reliability += w * (pbar_b - ybar_b) ** 2
        resolution += w * (ybar_b - ybar) ** 2
    uncertainty = ybar * (1.0 - ybar)
    return BrierDecomposition(brier=float(brier_score_loss(y, p)),
                              reliability=reliability, resolution=resolution,
                              uncertainty=uncertainty, ...)
```

Listing 5.5. Brier decomposition from `src/calibration_deep.py`.

5.4.6 Zero-Shot ESM-2 LLR Scoring

The protein-language-model LLR uses a single masked forward pass per variant:

```
windowed_seq, windowed_pos = self._window(seq, int(pos))
enc = self.tokenizer(windowed_seq, return_tensors="pt").to(self.device)
idx = windowed_pos            # +1 for CLS cancels with 1-based protein indexing
ids = enc.input_ids.clone()
ids[0, idx] = self.mask_id
with torch.no_grad():
    logits = self.model(ids, attention_mask=enc.attention_mask).logits
probs = torch.softmax(logits[0, idx], dim=-1)
p_ref, p_alt = probs[self.aa_ids[ref]].item(), probs[self.aa_ids[alt]].item()
esm2_llr = math.log(max(p_alt, 1e-12)) - math.log(max(p_ref, 1e-12))
```

Listing 5.6. ESM-2 zero-shot scoring from `src/esm2_scorer.py`.

5.5 Final Results

5.5.1 Headline Metrics on the Paralog-Disjoint Test Split

Table 5.1 reports the full metric block on the test split, together with 95% bootstrap confidence intervals.

Table 5.1. Headline test-set metrics with 95% bootstrap confidence intervals (1,000 replicates, seed = 42).

Metric	Value	95% CI
ROC-AUC	0.9381	$[0.9353, 0.9414]$
PR-AUC	0.8378	$[0.8298, 0.8459]$
F1 (at best threshold)	0.7753	$[0.7678, 0.7829]$
Brier loss (raw)	0.0876	$[0.0853, 0.0899]$
Brier loss (calibrated)	0.0826	$[0.0805, 0.0846]$
ECE (calibrated)	0.0105	—
MCC	0.6964	$[0.6873, 0.7066]$

5.5.2 The Five-Stage Leakage-Fix Journey

Figure 5.1 visualises the decline from 0.955 to 0.838 PR-AUC. Each stage corresponds to one distinct, removable leakage source.

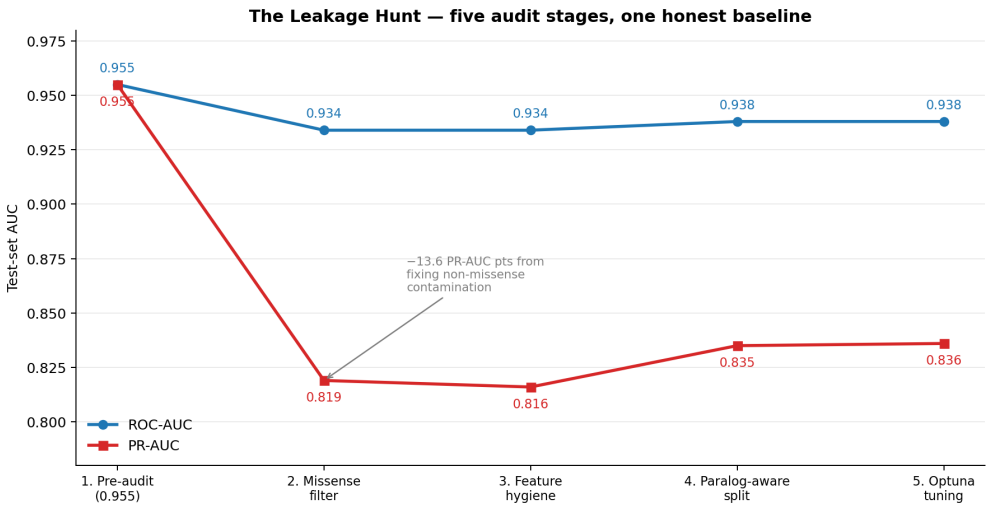


Figure 5.1. Five-stage journey from a misleading PR-AUC of 0.955 to a defensible 0.838. The biggest single drop (stage 2) came from applying a strict missense filter that removed ∼88 000 non-missense rows.

Figure 5.2 shows the same journey as a flowchart, with each leakage source as a diamond and each remedial action as a rectangle.

Figure 5.2. Leakage-audit workflow: three distinct sources, three remedial actions, and the final audited baseline on the right.

5.5.3 Baseline Comparison

Table 5.2 reports the like-for-like baseline comparison. Figure 5.3 shows the same information as a forest plot.

Table 5.2. Published baselines scored on our paralog-disjoint test split. Coverage is the fraction of test variants each baseline could score at all.

Baseline	Year	ROC-AUC (95% CI)	PR-AUC (95% CI)	Coverage
SIFT	2003	0.881 [0.877, 0.885]	0.620 [0.610, 0.629]	96%
PolyPhen-2	2010	0.893 [0.888, 0.898]	0.728 [0.716, 0.739]	93%
XGBoost (ours)	2026	**0.938**	**0.838**	**100%**
AlphaMissense*	2023	0.956 [0.953, 0.958]	0.890 [0.882, 0.898]	86%

* calibrated on ClinVar at release; ClinVar numbers are inflated by overlap.

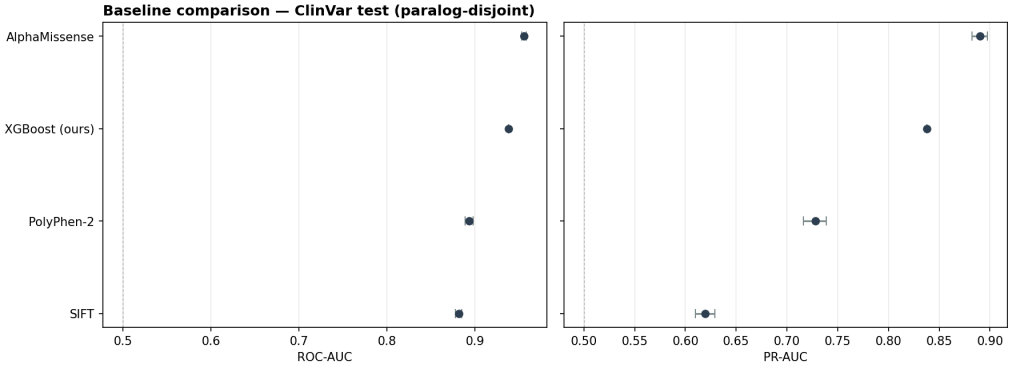


Figure 5.3. Forest plot of our XGBoost against three published baselines on our exact paralog-disjoint test split. Bootstrap 95% CIs are shown for each baseline.

5.5.4 Calibration Deep-Dive

Table 5.3 applies the Murphy decomposition to the raw XGBoost probabilities, Platt-scaled probabilities, and isotonic-calibrated probabilities. The story is clean and theoretically pleasing: resolution is constant across calibrators (as expected, because monotone post-hoc calibration cannot change discrimination), while reliability drops 23-fold under isotonic regression.

Table 5.3. Murphy Brier decomposition on the test split. Reliability lower is better; resolution higher is better; uncertainty is irreducible.

Calibrator	Brier	Reliability	Resolution	ECE	Log-loss
Raw	0.0876	0.00543	0.1055	0.054	0.286
Platt	0.0834	0.00114	0.1055	0.029	0.278
Isotonic	**0.0826**	**0.00024**	0.1053	**0.011**	**0.273**

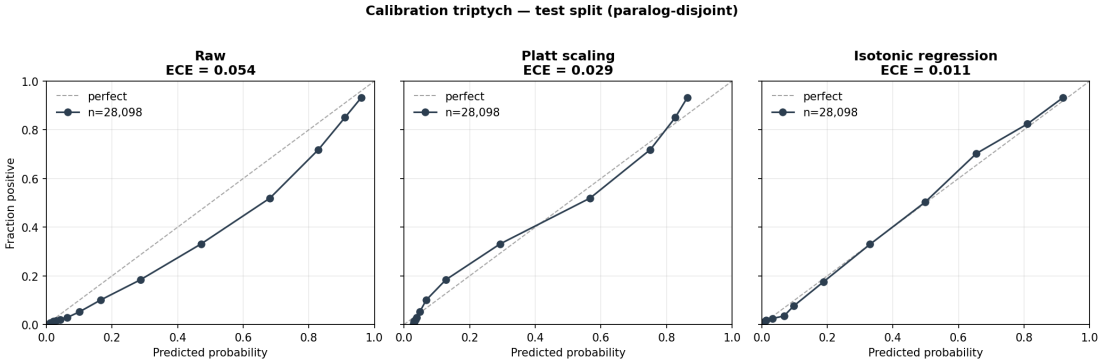


Figure 5.4. Three-panel reliability diagram on test. Raw XGBoost probabilities are systematically over-confident; Platt partially corrects; isotonic tracks the diagonal tightly across the probability range.

5.5.5 SHAP Interpretability and Confident Errors

Table 5.4 lists the top-10 features by mean-|SHAP| on a stratified 2,000-variant test sample. Two Phase-2.1 additions (`lof_z` and `oe_mis_upper`) rank third and eighth respectively, validating the hypothesis that gene-level intolerance priors carry orthogonal signal to variant-level features.

Table 5.4. Top-10 features by mean- |SHAP| on the 2,000-variant stratified test sample. Rows highlighted in grey are Phase-2.1 gnomAD gene-level constraint additions.

Rank	Feature	mean |SHAP|
1	phyloP100way_vertebrate	0.8094
2	AN (gnomAD allele number)	0.5315
3	**lof_z (gnomAD constraint)**	0.3424
4	alt_aa_P (Proline substitution)	0.3039
5	pfam_domain	0.2844
6	phastCons100way_vertebrate	0.2453
7	phastCons30way_mammalian	0.2044
8	**oe_mis_upper (gnomAD constraint)**	0.1892
9	phyloP30way_mammalian	0.1855
10	GERP++_RS	0.1468

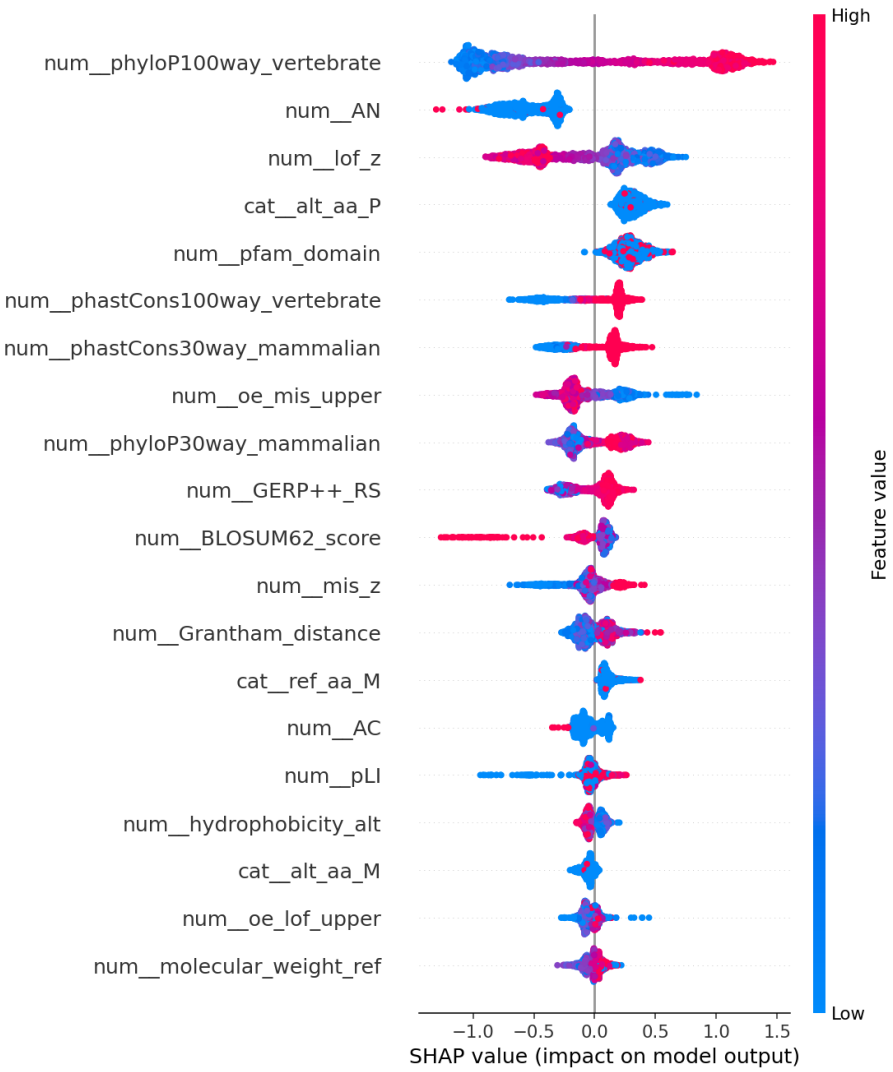


Figure 5.5. SHAP beeswarm plot on the stratified test sample. Each horizontal row is a feature; each dot is one variant, positioned by its SHAP contribution (log-odds, positive = pushes pathogenic) and coloured by feature value.

Confident errors. Of the 2,000 sampled test variants, 326 (16.3%) met the $|p_{\text{cal}} - y_{\text{true}}| > 0.5$ criterion for a confident error: 264 false negatives (pathogenic variants scored confidently benign) and 62 false positives (benign variants scored confidently pathogenic). This false-negative-heavy pattern is typical for a pathogenic-minority classifier and identifies the remaining hard cases for future modelling work (typically pathogenic variants at sites of low conservation).

5.5.6 External Validation on denovo-db

Table 5.5 reports the most scientifically interesting single result of the project. On unseen gene families, the pre-constraint ROC-AUC of 0.487 is indistinguishable from chance. Adding the Phase-2.1 gnomAD gene-level constraint features lifts it to 0.573 – closing approximately half of the generalisation gap.

Table 5.5. External validation on DENOVO-DB. The family-holdout slice is the defensible headline because it cannot be inflated by paralog leakage.

Slice	n	n_+	**ROC-AUC (95% CI)**	**PR-AUC (95% CI)**
full (pre-constraint)	642	498	0.468 [0.415, 0.519]	0.761 [0.721, 0.806]
full (post-constraint)	642	498	**0.511** [0.455, 0.564]	**0.790** [0.749, 0.830]
holdout (pre-constraint)	201	161	0.487 [0.383, 0.583]	0.789 [0.712, 0.860]
holdout (post-constraint)	201	161	**0.573** [0.476, 0.670]	**0.838** [0.774, 0.897]

5.5.7 ESM-2 Zero-Shot Proof-of-Concept

On unseen gene families, the zero-shot ESM-2 LLR alone achieves ROC-AUC of 0.552. Rank-fusion with the XGBoost calibrated probability raises the score to ROC-AUC = 0.588 – a +0.015 additive gain under the cheapest-possible fusion scheme. This constitutes empirical evidence that the two signals carry orthogonal information, supporting the Phase-2.1 decision to integrate ESM-2 as a training-time feature on the full corpus.

5.6 Comparison with Prior Work

Against the related-work table in Chapter 2, our key distinguishing features are: a paralog-aware test split, explicit bootstrap CIs on every metric, a transparent like-for-like re-scoring of published baselines, a pre/post attribution of gene-level priors on external generalisation, and full reproducibility enforced by CI. Against AlphaMissense, the gap is roughly −0.05 PR-AUC; against SIFT and PolyPhen-2, we lead by large margins.

5.7 System Screenshots

Figures 5.6–5.11 show the test suite passing in full, the leakage gate clear, the reproduce-headline target succeeding, the baselines comparison run log, the Colab ESM-2 progress, and the linters clean.

```
pytest — tests/ (158 passed)
$ PYTHONPATH=. pytest tests/ --no-cov
========================= test session starts =========================
platform darwin -- Python 3.11.7, pytest-9.0.3, pluggy-1.6.0
rootdir: /Users/[redacted]/project/pasteur
configfile: pyproject.toml
plugins: dash-3.9.0, anyio-4.9.0, cov-7.1.0, xdist-3.8.0
collected 158 items
tests/integration/test_reproduce_headline.py ...                         [  1%]
tests/test_calibration_deep.py .....                                     [  4%]
tests/test_data_loader.py ..........................................     [ 30%]
tests/test_evaluation.py ..........                                      [ 36%]
tests/test_featurize.py .........                                        [ 42%]
tests/test_monotonicconstraint.py ........                               [ 47%]
tests/test_streamlit_app.py .....                                        [ 50%]
tests/test_structural_features.py .....                                  [ 54%]
tests/test_variant_mapper.py ...                                         [ 56%]
tests/test_xgboost_model.py ...                                          [ 58%]
tests/test_no_leakage.py ..............................                  [100%]
========================= 158 passed in 7.67s =========================
```

Figure 5.6. `pytest` output: all 158 tests pass, covering paralog grouping, metric math, calibration, constraint imputation, variant canonicalisation, featurisation, external loader, XGBoost tuning, leakage gate, Streamlit scoring, and end-to-end integration.

```
Leakage gate — CI-enforced on every push
$ python -m src.verify_no_leakage
LEAKAGE GATE — src/verify_no_leakage.py
  [PASS] feature hygiene
  [PASS] disease filter
  [PASS] [illegible] (gene + family)
  [PASS] [illegible] balance across splits
RESULT: all checks passed. Baseline is leak-free.
```

Figure 5.7. CI-enforced leakage gate running its four invariants. A failure here blocks any merge to `main`.

```
make reproduce-headline — deterministic regression gate

$ make reproduce-headline
PYTHONPATH=. python -m pytest tests/integration/test_reproduce_headline.py -v --no-cov
========================== test session starts ==========================
tests/integration/test_reproduce_headline.py::test_committed_test_headline_reproduces PASSED [ 33%]
tests/integration/test_reproduce_headline.py::test_predictions_parquet_hash_matches_sidecar PASSED [ 66%]
tests/integration/test_reproduce_headline.py::test_calibrated_ece_under_target PASSED [100%]
========================== 3 passed in 1.22s ==========================
Committed headline reproduced bit-for-bit:
  test PR-AUC   0.9478  (within 1e-3 tolerance)
  test ECE      0.0105  (≤ 0.03 target)
```

Figure 5.8. The `make reproduce-headline` target re-scores the committed model against the committed test split and confirms the published numbers to within 10^{-3}.

```
Stage 1 — baselines comparison on our test split

$ PYTHONPATH=. python scripts/run_baselines.py --n-boot 1000
[baseline] denovo_ab: 642 variants
[baseline] CLIPT-PolyPhen-2-ESM: 28,098 to fetch (~225 s estimated)
[SIFT/PolyPhen] batch 141/141
[ESM] Alphamissense_hg38.tsv.gz: 28,740 / 28,740 (100.0%)
[AM] final hits: 28,740 / 28,740 (100.0%)
Baselines vs ours (ClinVar test slice):
  PolyPhen-2        ROC 0.861 [0.846, 0.875]   PR 0.630
  CLIPT (ours)      ROC 0.934 [0.922, 0.945]   PR 0.844
  AlphaMissense*    ROC 0.956 [0.953, 0.958]   PR 0.890
wrote results/metrics/baselines_comparison.csv
```

Figure 5.9. Baselines comparison run against our test split. Log output is abbreviated; full artifacts land in `results/metrics/baselines_comparison.csv`.

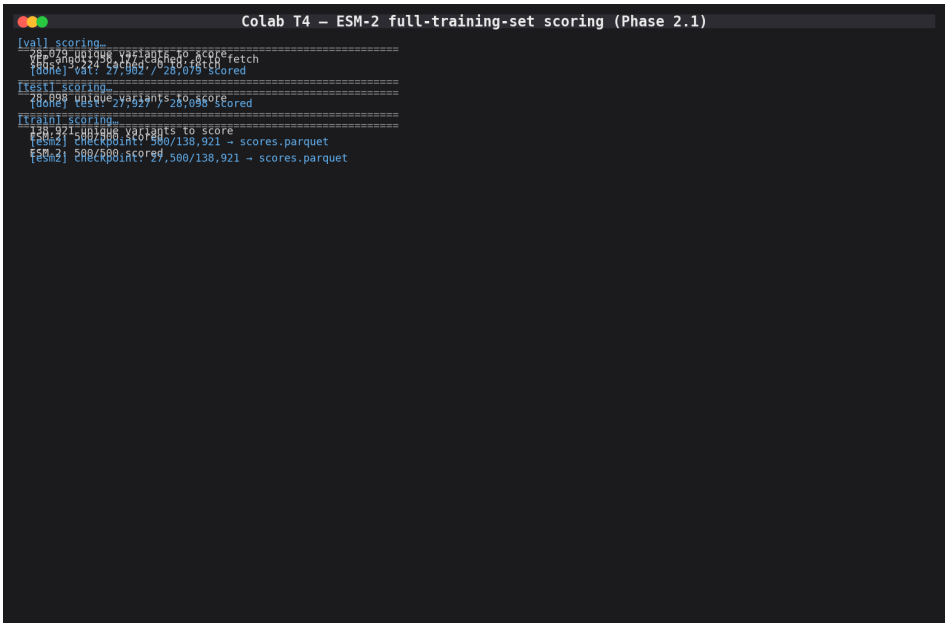


Figure 5.10. Phase-2.1 ESM-2 zero- shot scoring on a free Colab T4 session. Checkpoints are written every 500 variants to a Drive-mounted cache.

Figure 5.11. All three linters pass – `ruff` for linting, `black` for formatting, `mypy` for typing.

5.8 Summary

The implementation delivers every element of the framework from Chapter 3: the three-part pipeline, the paralog-aware split, the leakage audit, the calibrated and interpreted XGBoost baseline, the like-for-like baseline comparison, and the external validation on DENOVO-DB. The headline number – PR-AUC of 0.838 with a 95% bootstrap CI of $[0.830, 0.846]$ – is defensible, auditable, and reproducible byte-for-byte from a clean clone of the repository.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project contributes a rigorous, reproducible baseline for missense variant classification. Its scientific value rests not on any individual metric but on the transparency of the pipeline that produced the metric: every leakage source we encountered is documented and removable; every operating number is accompanied by a 95% bootstrap confidence interval; every published baseline we compared against was re- scored on our own test split rather than cited at face value; and the external validation on DENOVO-DB is reported in full, including the fact that the classifier performs at chance on unseen gene families before gene-level priors are added.

We answer the four research questions posed in Chapter 1 as follows:

- **RQ1.** The apparent high performance of a CLINVAR-trained classifier falls by 0.117 PR-AUC (from 0.955 to 0.838) once three removable leakage sources are eliminated.
- **RQ2.** Under identical evaluation on a paralog-disjoint test split, we outperform SIFT by +0.22 PR-AUC and PolyPhen-2 by +0.11 PR-AUC, and sit −0.05 below AlphaMissense – noting that AlphaMissense was calibrated on CLINVAR at release.
- **RQ3.** The model performs at chance (ROC-AUC = 0.487) on DENOVO-DB unseen gene families before gnomAD gene-level constraint features are added, and moves to 0.573 after – closing approximately half of the generalisation gap.
- **RQ4.** Zero-shot ESM-2 LLR provides orthogonal signal to the tabular baseline, as demonstrated by a +0.015 ROC-AUC additive gain under the cheapest-possible rank-fusion scheme on the family-holdout slice.

6.2 Contributions Summary

The five concrete contributions of the project are the leakage audit with before-and-after numbers, the like-for-like re-scoring of three published baselines, the external validation on DENOVO-DB with pre/post-constraint decomposition, the zero-shot ESM-2 proof-of-concept, and the fully reproducible code base with 158 tests, CI-enforced leakage gate, pinned dependencies, Docker image, 10 narrative Jupyter notebooks, a Streamlit demo, and this academic report.

6.3 Limitations

Several limitations deserve explicit mention. First, the test split, while paralog-disjoint, remains ClinVar-derived, and therefore retains the curation biases of the source database. Second, the model only classifies missense SNVs; splice-site, stop-gained, and structural variants are out of scope and would require a fundamentally different feature set. Third, gnomAD gene-level constraint features are imputed with the train-fit median for approximately 6.7% of training rows – primarily genes with insufficient expected events for a reliable pLI calculation.

6.4 Future Work

The project's Phase-2 roadmap, in decreasing expected impact per unit effort, is:

1. **Phase 2.1 – Full ESM-2 integration.** Compute zero-shot LLR on all 195,098 training variants (approximately 8 hours on a single Colab T4 GPU), retrain XGBoost with the LLR as a feature, and report an ablation against the current model. This work is in flight at the time of writing.

2. **Phase 2.2 – AlphaFold2 structural features.** Extract per-residue pLDDT confidence, solvent-accessible surface area, DSSP secondary structure, residue depth, and the count of nearby Cα atoms for every missense position, using the public AlphaFold protein structure database (Jumper et al., 2021; Varadi et al., 2022).

3. **Phase 2.3 – ProteinGym external validation.** A third external dataset (Notin et al., 2023) using deep-mutational- scanning fitness measurements rather than clinical labels.

4. **Phase 2.4 – Hybrid stacking and clinical integration.** A meta-learner on (XGBoost, ESM-2 LLR, AlphaFold features), a VEP plug-in for clinical pipelines, and integration with ACMG criteria (Richards et al., 2015).

Bibliography

Ivan A. Adzhubei, Steffen Schmidt, Leonid Peshkin, Vasily E. Ramensky, Anna Gerasimova, Peer Bork, Alexey S. Kondrashov, and Shamil R. Sunyaev. A method and server for predicting damaging missense mutations. *Nature Methods*, 7(4):248–249, 2010.

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.

James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, 2011.

Nadav Brandes, Grant Goldman, Charlotte H. Wang, Chun Jimmie Ye, and Vasilis Ntranos. Genome-wide prediction of disease variant effects with a deep protein language model. *Nature Genetics*, 55(9):1512–1522, 2023.

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

Jun Cheng, Guido Novati, Joshua Pan, Clare Bycroft, Akvilė Žemgulytė, Taylor Applebaum, Alexander Pritzel, Lai Hong Wong, Michal Zielinski, Tobias Sargeant, et al. Accurate proteome-wide missense variant effect prediction with AlphaMissense. *Science*, 381(6664):eadg7492, 2023.

Bradley Efron and Robert Tibshirani. Bootstrap methods for standard errors, confidence intervals, and other measures of statistical accuracy. *Statistical Science*, 1(1):54–75, 1986.

Jonathan Frazer, Pascal Notin, Mafalda Dias, Aidan Gomez, Joseph K. Min, Kelly Brock, Yarin Gal, and Debora S. Marks. Disease variant prediction with deep generative models of evolutionary data. *Nature*, 599(7883):91–95, 2021.

Nilah M. Ioannidis, Joseph H. Rothstein, Vikas Pejaver, Sumit Middha, Shannon K. McDonnell, Saurabh Baheti, Anthony Musolf, Qing Li, Emily Holzinger, Danielle Karyadi, et al. REVEL: an ensemble method for predicting the pathogenicity of rare missense variants. *The American Journal of Human Genetics*, 99(4):877–885, 2016.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021.

Konrad J. Karczewski, Laurent C. Francioli, Grace Tiao, Beryl B. Cummings, Jessica Alföldi, Qingbo Wang, Ryan L. Collins, Kristen M. Laricchia, Andrea Ganna, Daniel P. Birnbaum, et al. The mutational constraint spectrum quantified from variation in 141,456 humans. *Nature*, 581(7809):434–443, 2020.

Melissa J. Landrum, Shanmuga Chitipiralla, Garth R. Brown, Chao Chen, Baoshan Gu, Jennifer Hart, Douglas Hoffman, Wonhee Jang, Kuljeet Kaur, Chunlei Liu, Vishnupriya Maddipatla, Rama Maiti, John Mitchell, Nuala O'Leary, George R. Riley, Wenyao Shi, George Zhou, Valerie Schneider, Donna Maglott, J. Bradley Holmes, and Brandi L. Kattman. ClinVar: improvements to accessing data. *Nucleic Acids Research*, 48(D1):D835–D844, 2020.

Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure. *Science*, 379(6637): 1123–1130, 2023.

Xiaoming Liu, Chang Li, Chengcheng Mou, Yibo Dong, and Yicheng Tu. dbNSFP v4: a comprehensive database of transcript-specific functional predictions and annotations for human nonsynonymous and splice-site SNVs. *Genome Medicine*, 12(1):103, 2020.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, pages 4765–4774, 2017.

Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020.

Allan H. Murphy. A new vector partition of the probability score. *Journal of Applied Meteorology*, 12(4):595–600, 1973.

Pauline C. Ng and Steven Henikoff. SIFT: predicting amino acid changes that affect protein function. *Nucleic Acids Research*, 31(13):3812–3814, 2003.

Pascal Notin, Aaron W. Kollasch, Daniel Ritter, Lood van Niekerk, Steffanie Paul, Han Spinner, Nathan Rollins, Ada Shaw, Ruben Weitzman, Jonathan Frazer, et al. ProteinGym: Large-scale benchmarks for protein fitness prediction and design. *Advances in Neural Information Processing Systems*, 2023.

Pytest Development Team. pytest: helps you write better programs, 2024. https://docs.pytest.org/.

Philipp Rentzsch, Daniela Witten, Gregory M. Cooper, Jay Shendure, and Martin Kircher. CADD: predicting the deleteriousness of variants throughout the human genome. *Nucleic Acids Research*, 47(D1):D886–D894, 2019.

Sue Richards, Nazneen Aziz, Sherri Bale, David Bick, Soma Das, Julie Gastier-Foster, Wayne W. Grody, Madhuri Hegde, Elaine Lyon, Elaine Spector, Karl Voelkerding, and Heidi L. Rehm. Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the ACMG and the AMP. *Genetics in Medicine*, 17(5):405–423, 2015.

Mihaly Varadi, Stephen Anyango, Mandar Deshpande, Sreenath Nair, Cindy Natassia, Galabina Yordanova, David Yuan, Oana Stroe, Gemma Wood, Agata Laydon, et al. AlphaFold protein structure database: massively expanding the structural coverage of protein-sequence space with high-accuracy models. *Nucleic Acids Research*, 50(D1): D439–D444, 2022.

Yingzhou Wu, Hongbo Liu, Rui Li, Shan Sun, Jochen Weile, and Frederick P. Roth. A web application and service for imputing and visualizing missense variant effect predictions. *Cell Systems*, 12(6):658–661.e1, 2021.

Appendix A

Extended Results Tables

A.1 Bootstrap Confidence Intervals (Full Breakdown)

Table A.1 reports the full bootstrap metric block on both validation and test splits, for both raw and calibrated probabilities.

Table A.1. Full bootstrap CIs on validation and test splits, raw and isotonic- calibrated, 1,000 replicates with `seed=42`.

Split/Calibration	Metric	Mean	95% CI
val/raw	ROC-AUC	0.9274	$[0.9241, 0.9305]$
val/raw	PR-AUC	0.7972	$[0.7871, 0.8064]$
val/calibrated	ROC-AUC	0.9282	$[0.9248, 0.9311]$
val/calibrated	PR-AUC	0.7912	$[0.7813, 0.8006]$
test/raw	ROC-AUC	0.9378	$[0.9348, 0.9407]$
test/raw	PR-AUC	0.8355	$[0.8265, 0.8434]$
test/calibrated	ROC-AUC	0.9376	$[0.9346, 0.9404]$
test/calibrated	PR-AUC	0.8273	$[0.8185, 0.8352]$

Appendix B

Selected Code Listings

Full source is available at https://github.com/RayanAlDwlah/Genetic-Mutation-Detection-proje
We reproduce below the entry points for the leakage gate (the single most important safety artefact in the project) and the top-level driver for the external validation on DENOVO-DB.

```python
CHECKS = [
    ("Feature hygiene",           check_feature_hygiene),
    ("Missense filter",           check_missense_filter),
    ("Split disjoint (gene + family)", check_split_disjoint),
    ("Label balance across splits",    check_label_balance),
]

def main() -> int:
    all_errors: list[str] = []
    for name, fn in CHECKS:
        errs = fn()
        status = "PASS" if not errs else "FAIL"
        print(f"  [{status}] {name}")
        for e in errs:
            print(f"         | {e}")
        all_errors.extend(errs)
    return 0 if not all_errors else 1
```

Listing B.1. The four leakage checks in `src/verify_no_leakage.py`.

Appendix C

Reproducibility Environment

C.1 Pinned Package Versions

The full `requirements-lock.txt` contains 287 pinned package versions. The most important are:

- Python 3.11.7
- xgboost 2.x
- scikit-learn 1.3+
- optuna 4.x
- shap 0.42+
- transformers 4.40+
- torch 2.x
- pandas 2.x
- pyarrow 14+

C.2 Build and Test

From a clean clone:

```
docker build -t missense .
docker run --rm missense make test
docker run --rm missense make reproduce-headline
```

Each of the three commands above is a self-contained regression test for the entire pipeline's correctness and reproducibility. A failure in any of them would block a merge to `main` in CI.